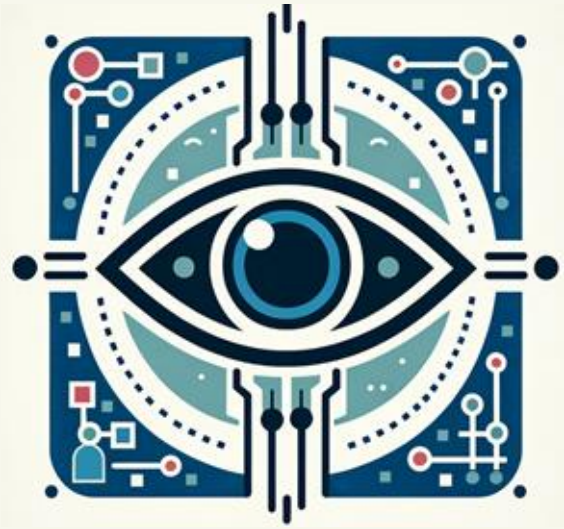
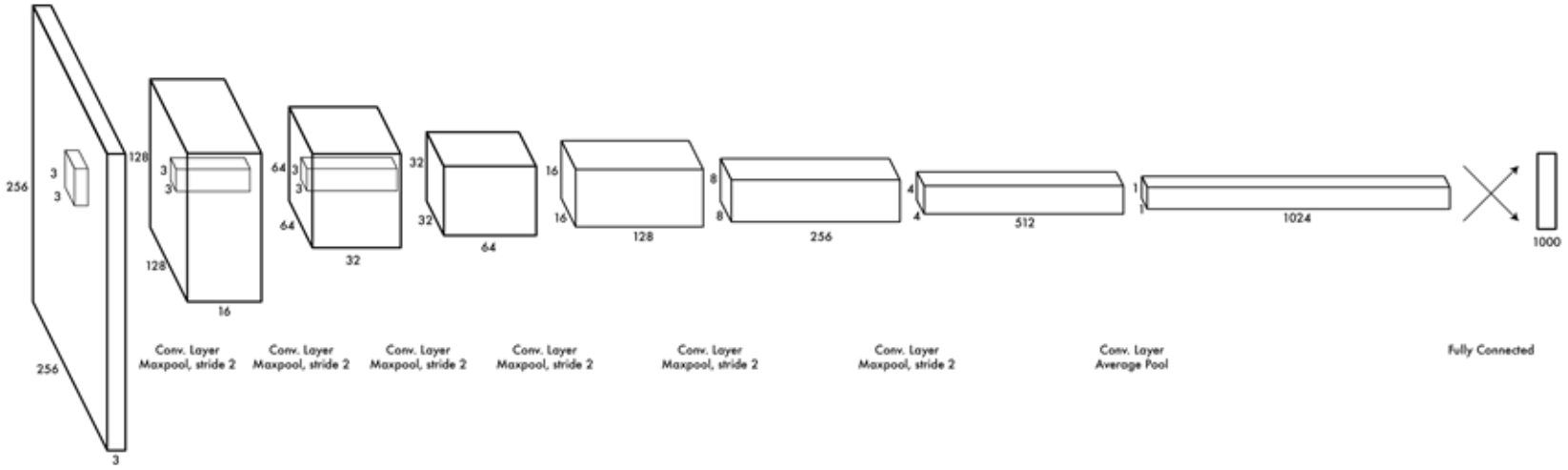


COMPUTER VISION

Structured Predictions with Convolutional Neural Networks

Luis F. Teixeira | M.EIC+M.IA | FEUP

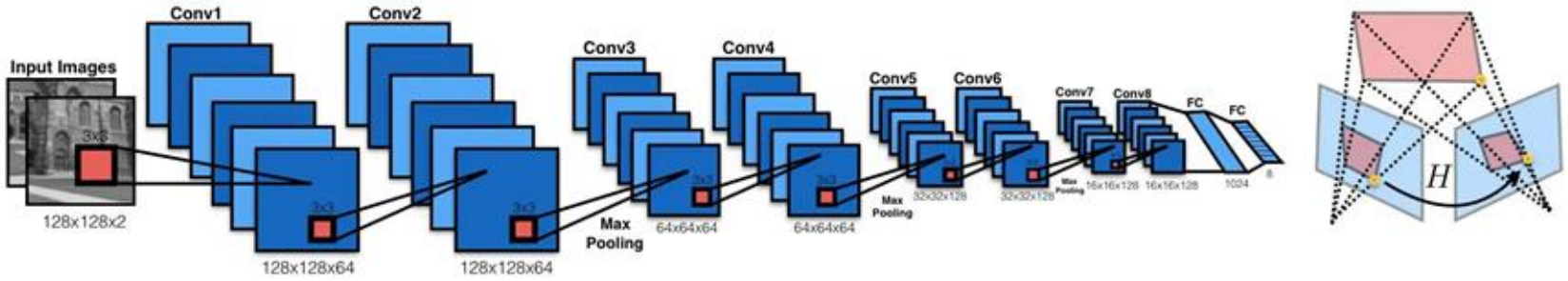
Beyond classification

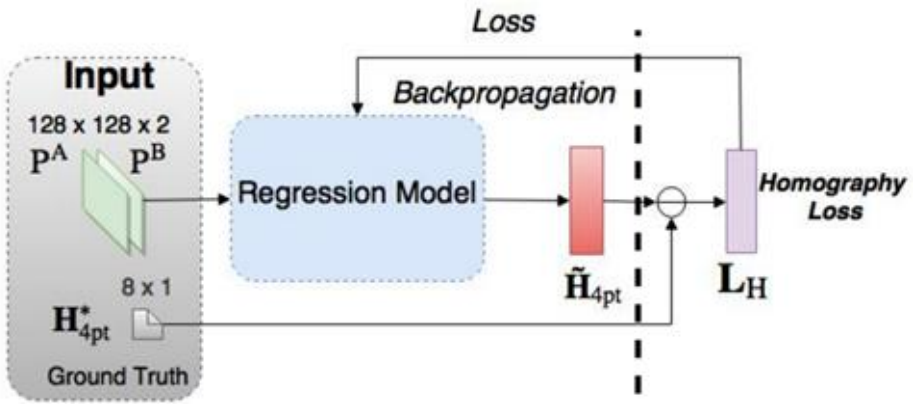


class probabilities

other types of outputs?

CNN for homography calculation



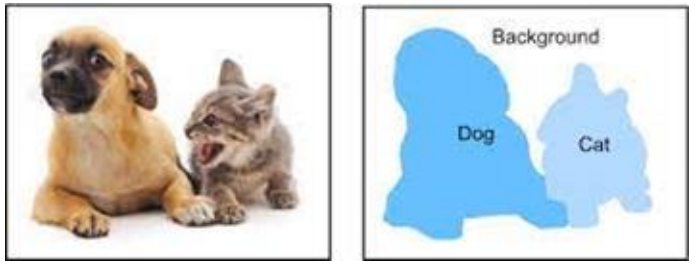


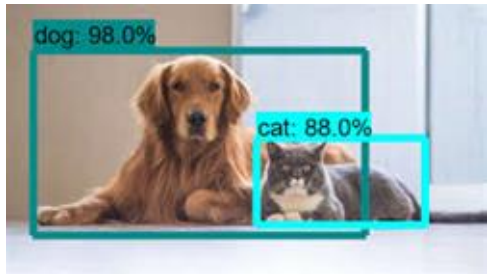
- eight real numbers as output
- supervised learning is done by the loss between the output and the ground truth homography

DeTone et al.

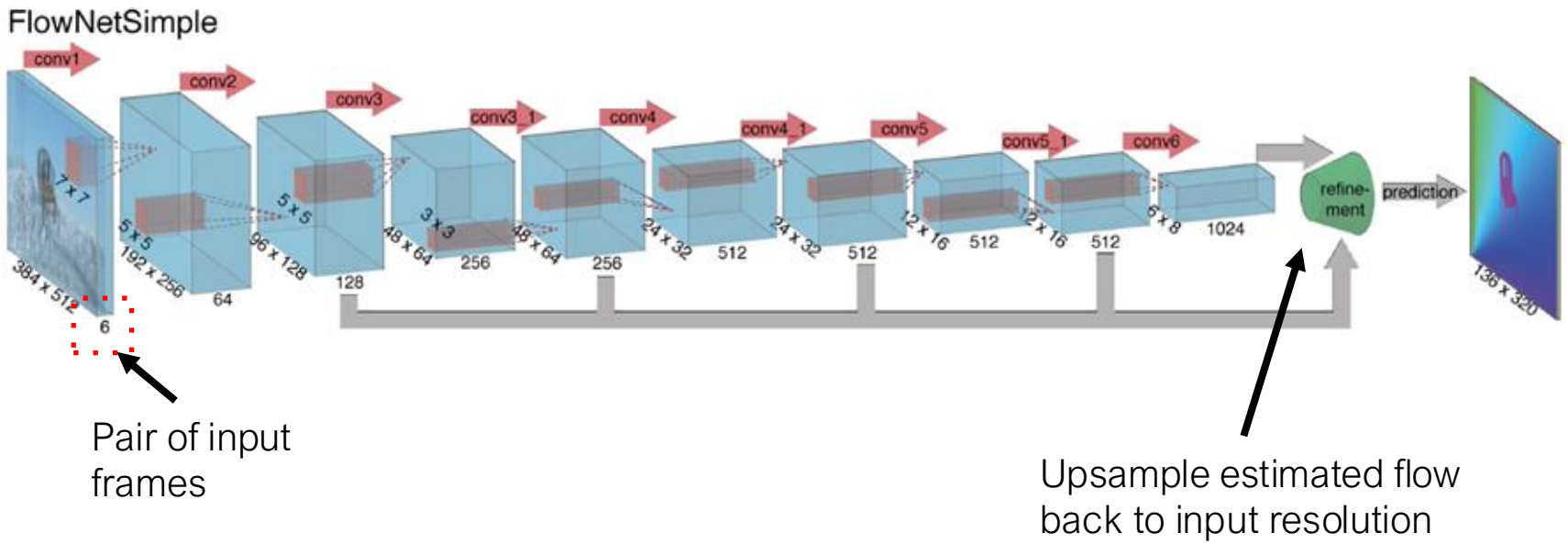
More complex outputs

- Attributes
- Image Output
  - e.g. colorization, semantic segmentation, super-resolution, stylization, depth estimation...)
- Object Detections
- Semantic Keypoints
- Text Captions



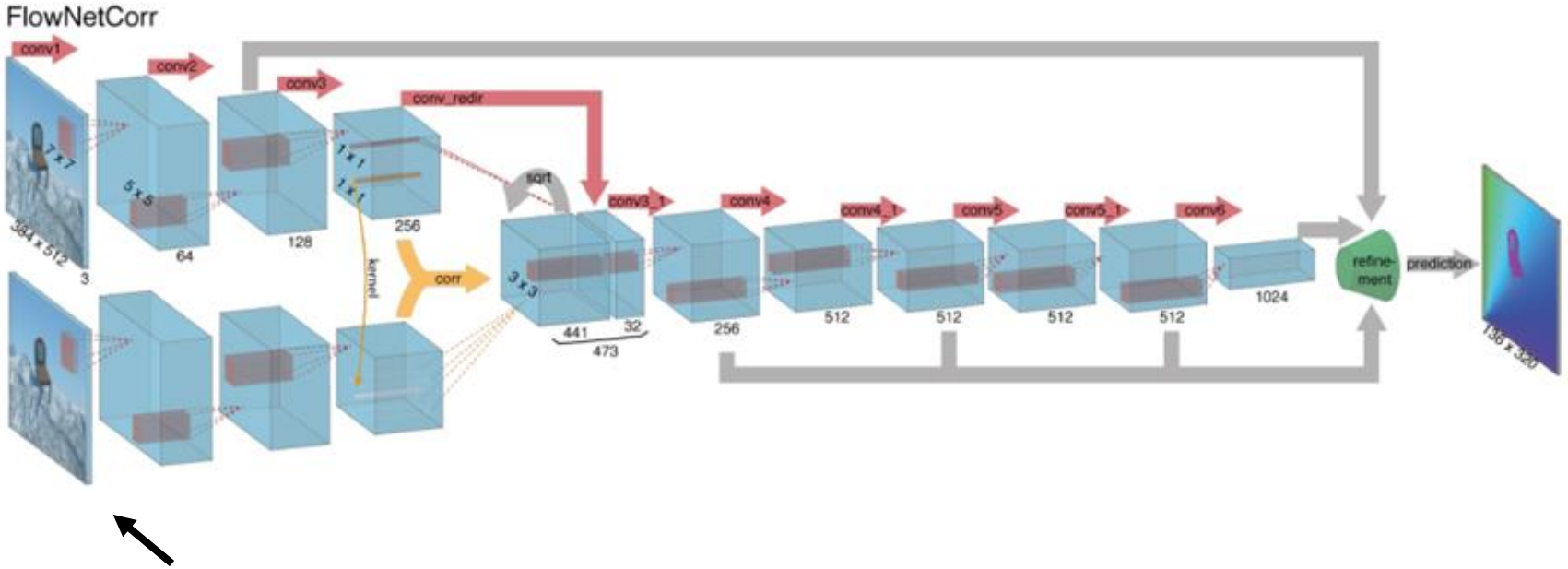


CNN to estimate optical flow



Fischer et al. 2015. https://arxiv.org/abs/1504.06852

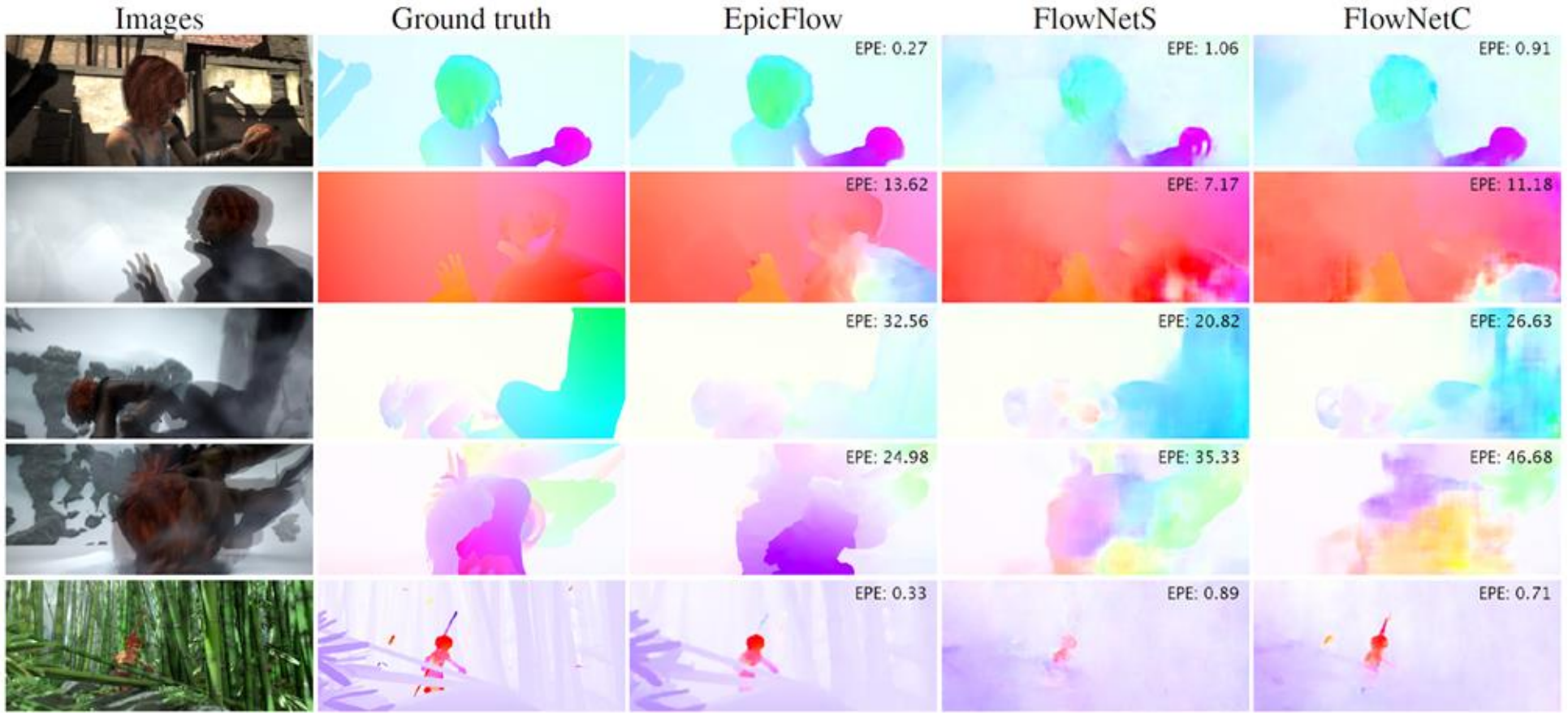
CNN to estimate optical flow



Alternatively, extract features separately and combine later

CNN to estimate optical flow

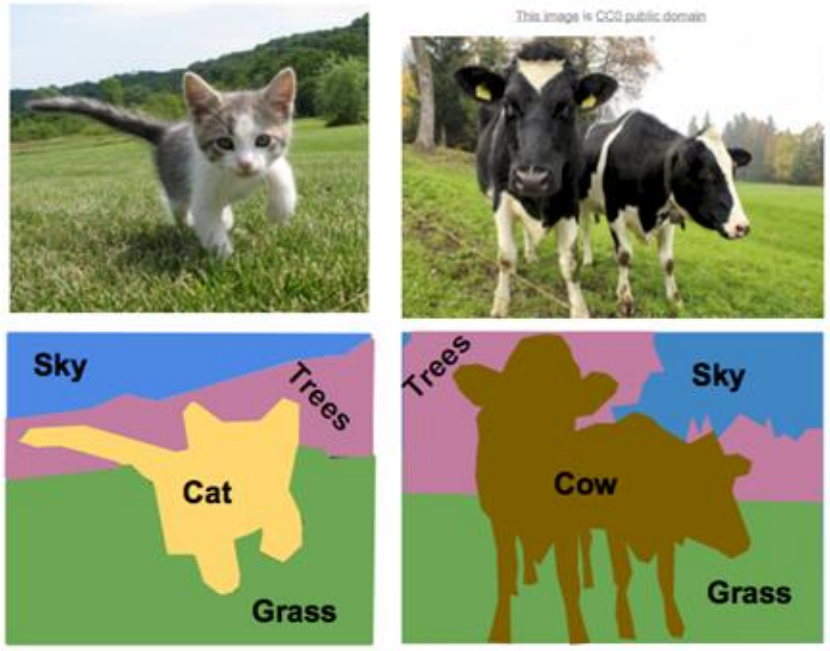
Results on Sintel (open source 3D animated short film)



Fischer et al. 2015. https://arxiv.org/abs/1504.06852

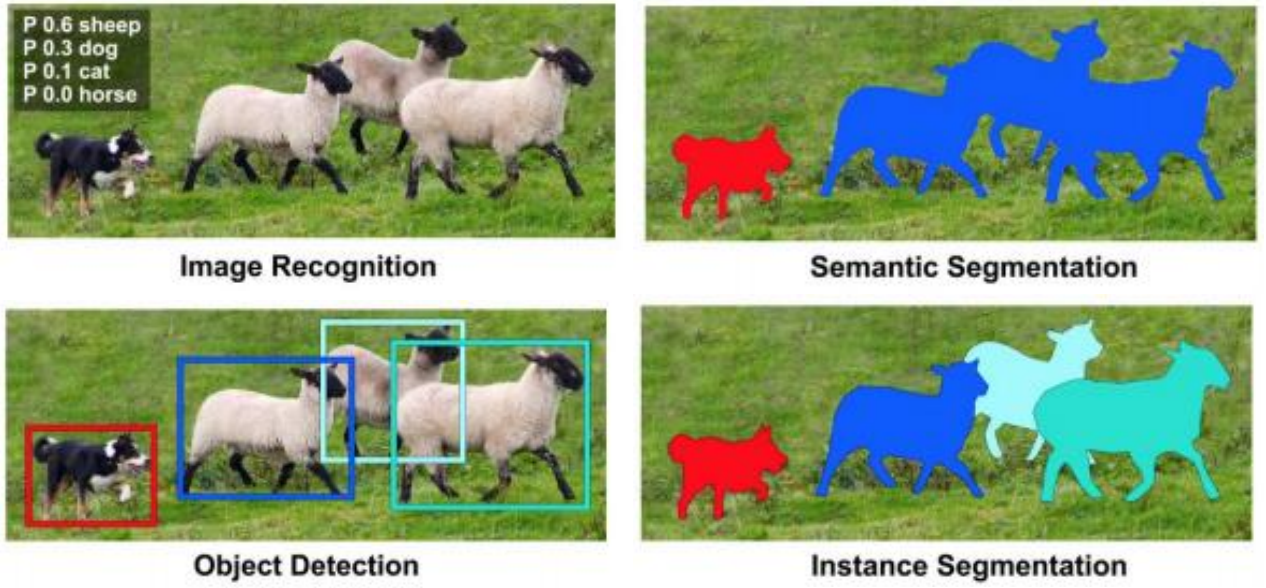
Segmentation

- Image segmentation partitions the image into meaningful regions
  → predict a label at every pixel



Segmentation

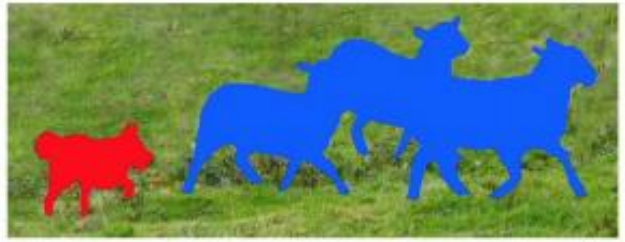
- **Semantic segmentation** assigns a pixel value or label to every concept in the image
- **Instance segmentation** assigns also a numeric value for every instance of the same semantic value



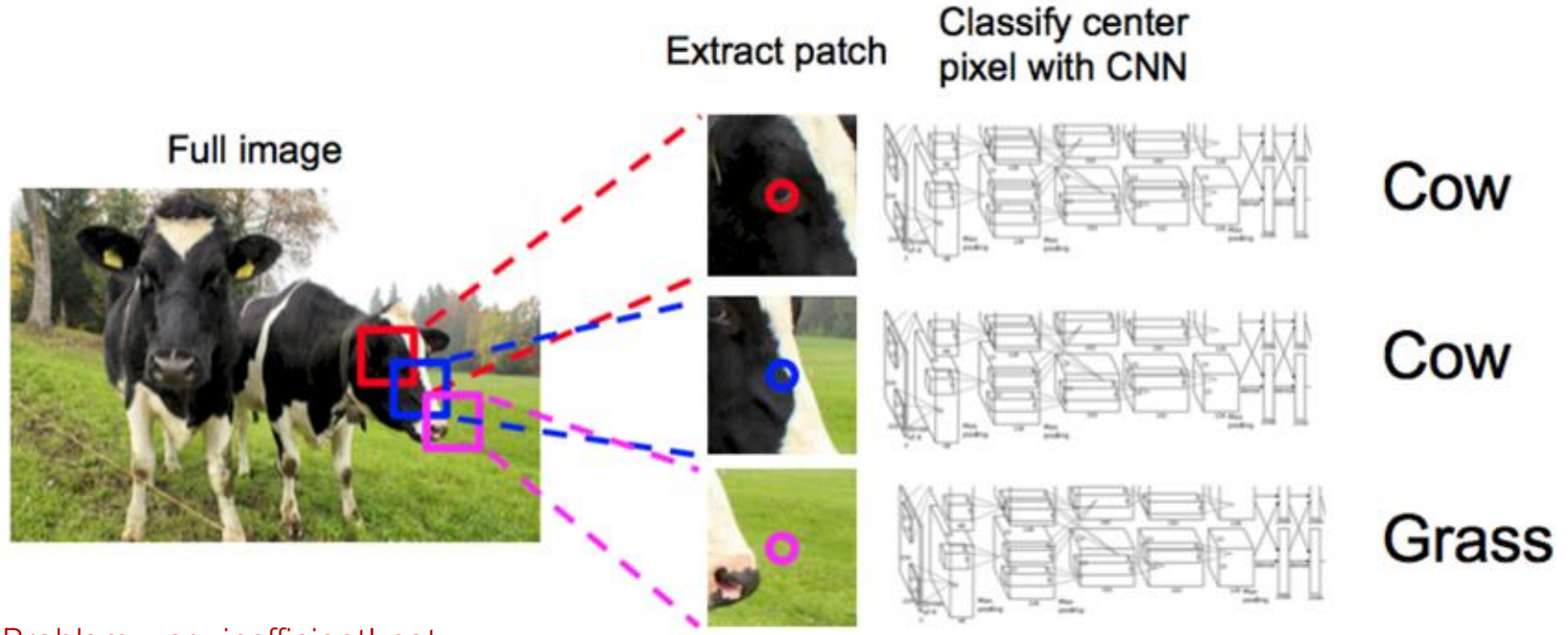
Semantic Segmentation

- How can we do semantic segmentation?
  - Until now a network gives us a prediction for the whole image

vs.



Sliding Window

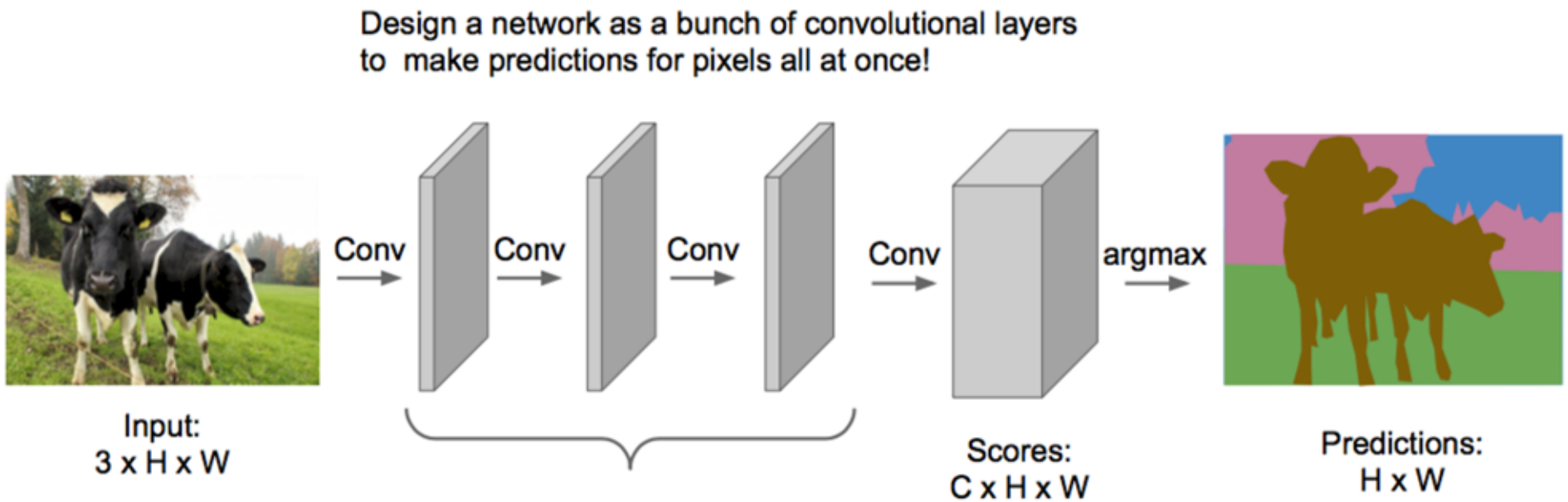


Problem: very inefficient! not reusing shared features between overlapping patches

Farabet et al, "Learning Hierarchical Features for Scene Labeling," TPAMI 2013
Pinheiro and Collobert, "Recurrent Convolutional Neural Networks for Scene Labeling", ICML 2014

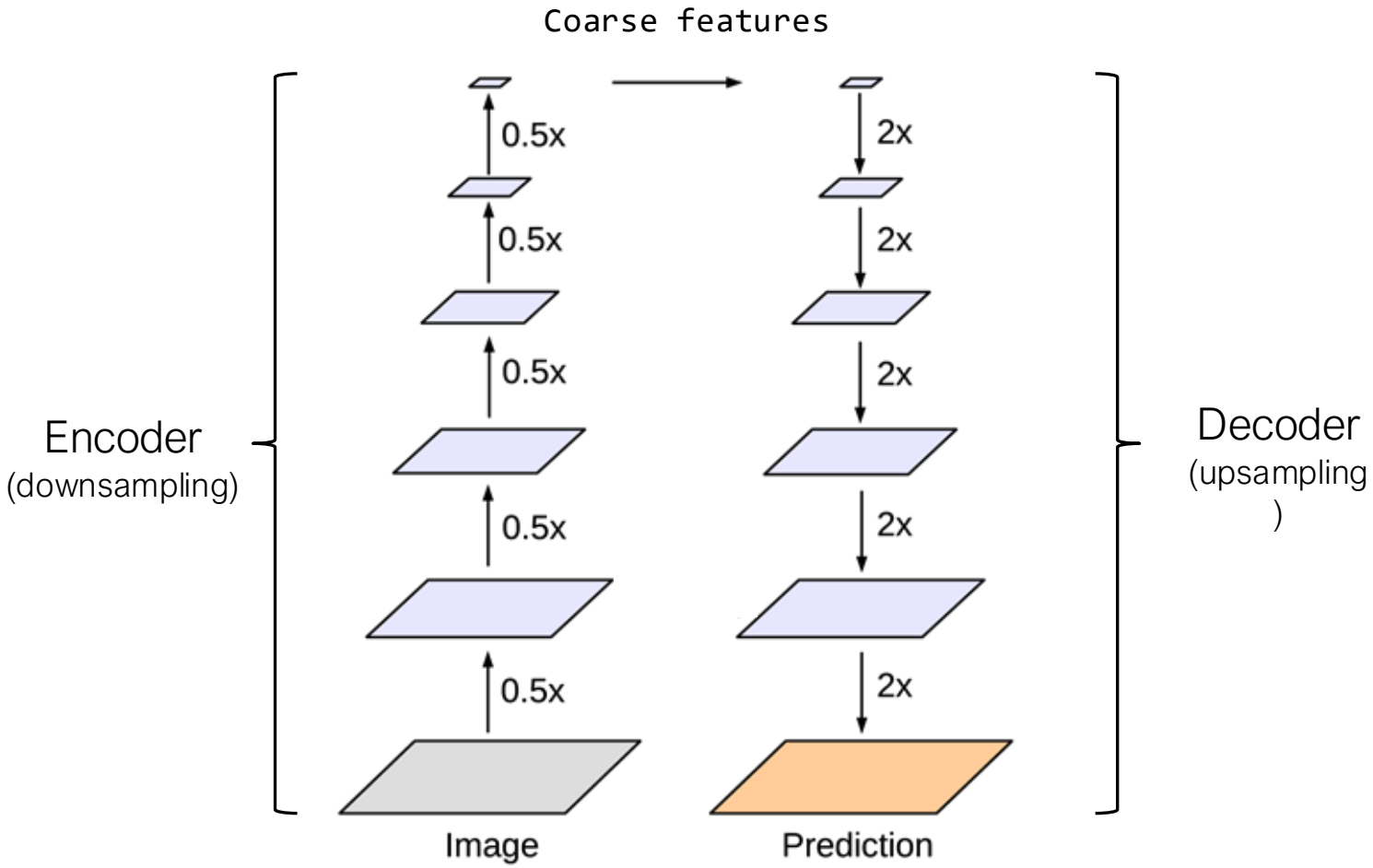
Fully Convolutional Network (FCN)

(no fully connected layer is used)



Problem: convolutions at original image resolution will also be very expensive...

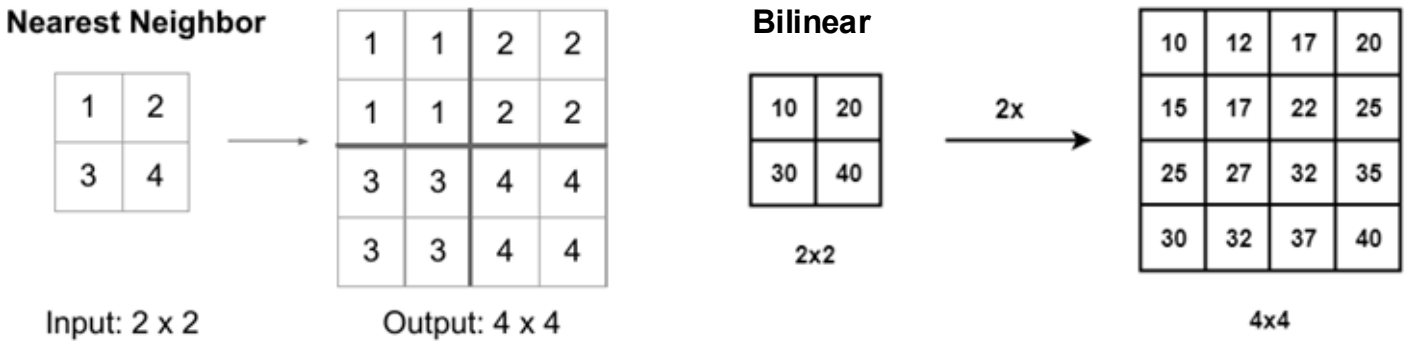
Fully Convolutional Network (FCN)



How can we do the decoding/upsampling?

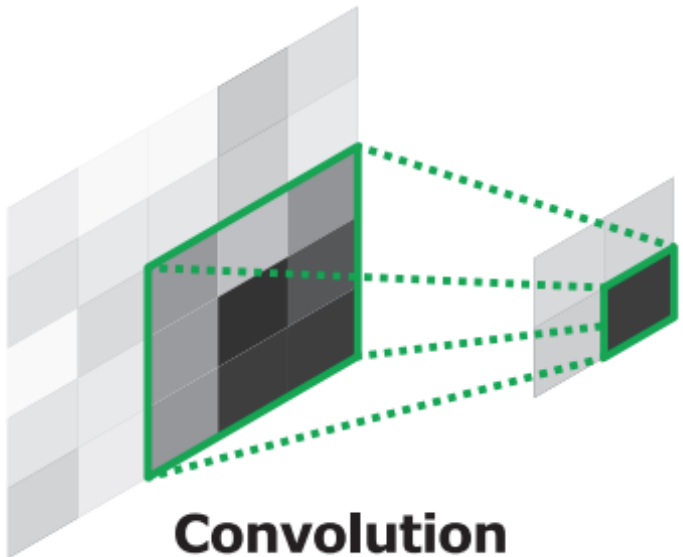
Upsampling

- Any kind of classical interpolation would be acceptable: nearest-neighbor, bilinear, bicubic, etc.

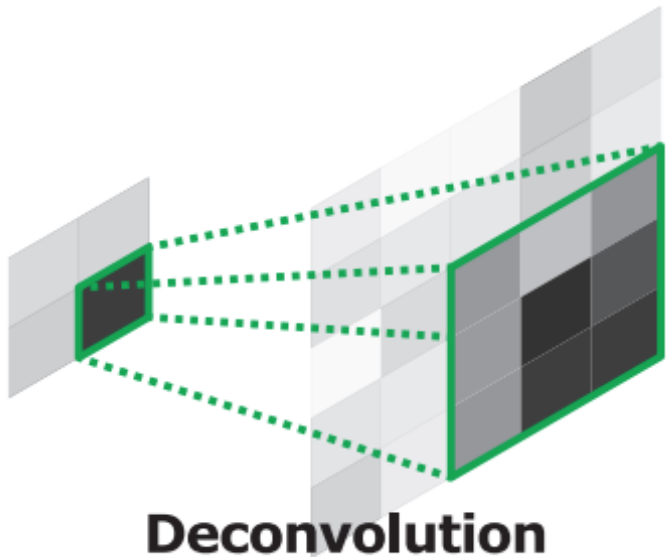


- However, the most common approach in deep learning is **transposed convolution** (sometimes called deconvolution)

  → The decoder will also be learned



Convolution

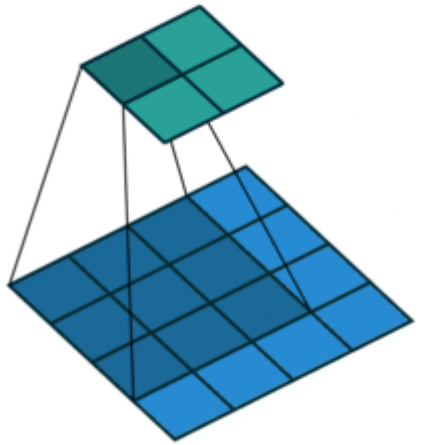


Deconvolution

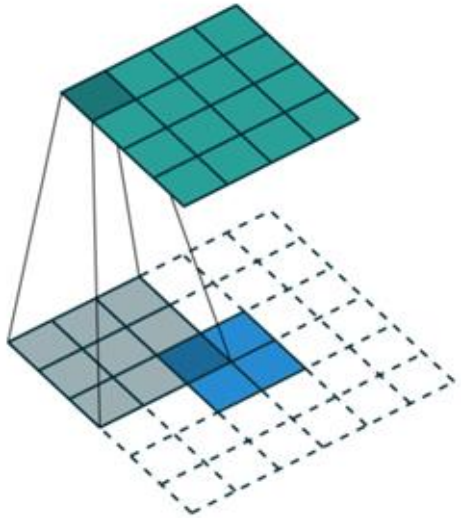
Image source: https://arxiv.org/pdf/1505.04366.pdf

Learnable Upsampling - Transposed Convolution

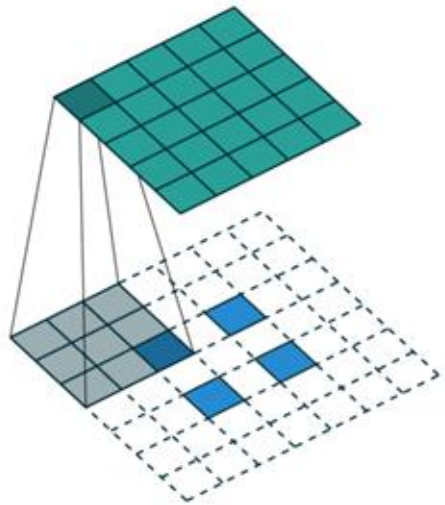
Convolution



Transposed convolution = padding/striding smaller image then weighted sum of input x filter: ‘stamping’ kernel



2x2, stride 1, 3x3 kernel,
upsample to 4x4



2x2, stride 2, 3x3 kernel,
upsample to 5x5.

Image Source: https://towardsdatascience.com/types-of-convolutions-in-deep-learning-717013397f4d

Kernel

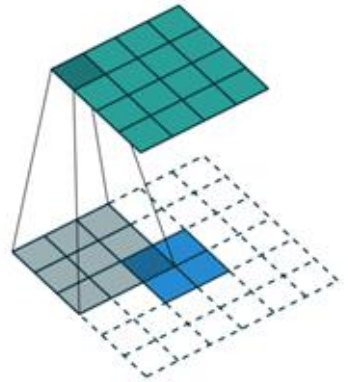
1	1	1
1	1	1
1	1	1

Feature map

1	2
3	4

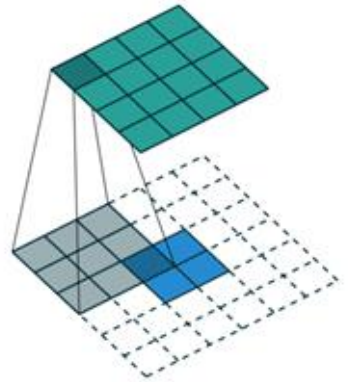
Padded feature map

		1	2		
		3	4		



Kernel

1	1	1
1	1	1
1	1	1



Input feature map

1	2
3	4

Output feature map

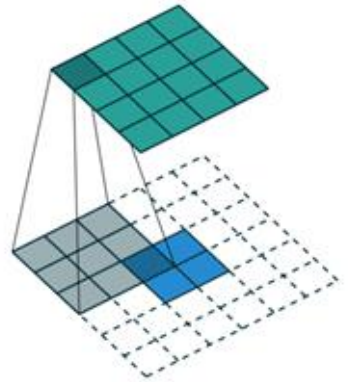
1	1	1			
1	1	1			
1	1	1			

Padded input feature map

		1	2		
		3	4		

Kernel

1	1	1
1	1	1
1	1	1



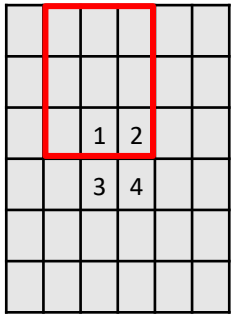
Input feature map

1	2
3	4

Output feature map

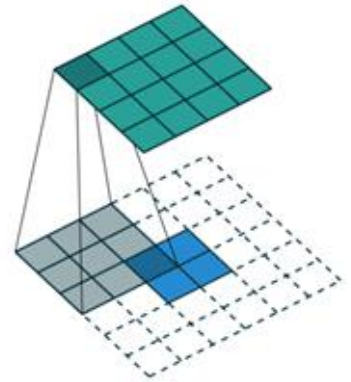
1	4	4	3		
1	4	4	3		
1	4	4	3		

Padded input feature map



Kernel

1	1	1
1	1	1
1	1	1



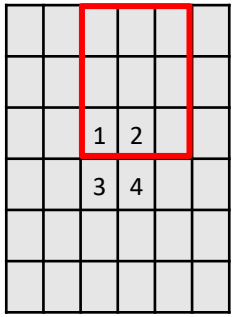
Input feature map

1	2
3	4

Output feature map

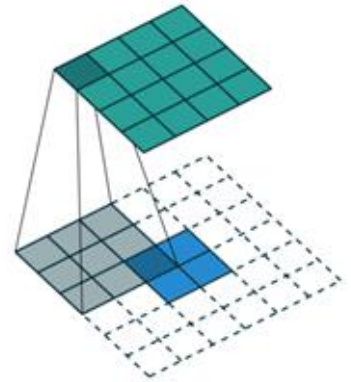
1	4	7	6	3	
1	4	7	6	3	
1	4	7	6	3	

Padded input feature map



Kernel

1	1	1
1	1	1
1	1	1



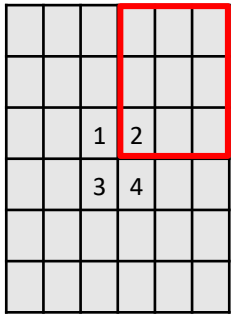
Input feature map

1	2
3	4

Output feature map

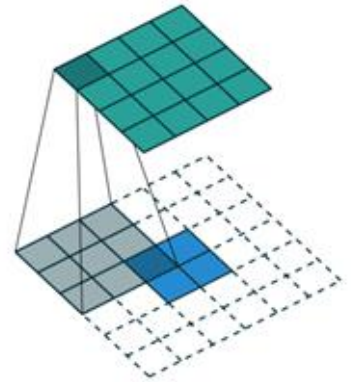
1	4	7	8	5	2
1	4	7	8	5	2
1	4	7	8	5	2

Padded input feature map



Kernel

1	1	1
1	1	1
1	1	1



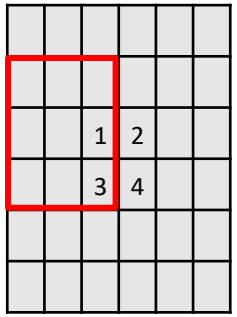
Input feature map

1	2
3	4

Output feature map

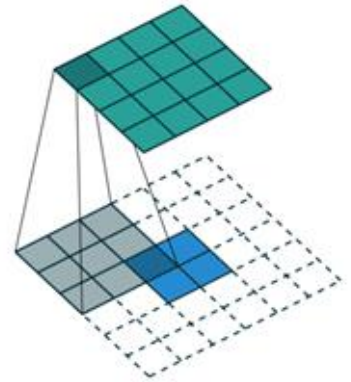
1	4	7	8	5	2
5	8	11	8	5	2
5	8	11	8	5	2
4	4	4			

Padded input feature map



Kernel

1	1	1
1	1	1
1	1	1



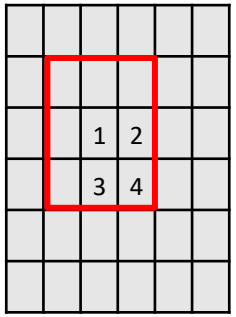
Input feature map

1	2
3	4

Output feature map

1	4	7	8	5	2
5	18	21	18	5	2
5	18	21	18	5	2
4	14	14	10		

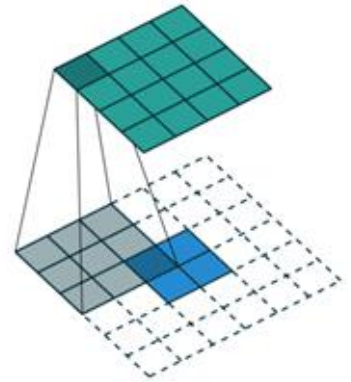
Padded input feature map



James Tompkin (inspired by andriys)

Kernel

1	1	1
1	1	1
1	1	1



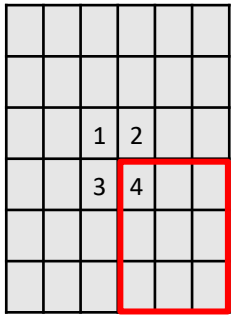
Input feature map

1	2
3	4

Output feature map

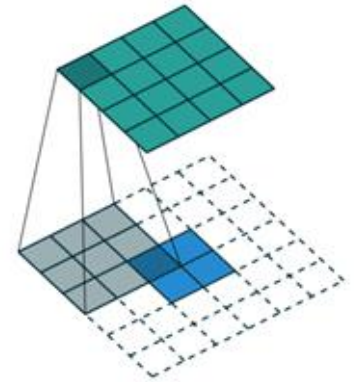
1	4	7	8	5	2
5	18	31	34	21	8
9	32	55	60	37	14
11	38	66	64	43	16
7	24	41	44	27	10
3	10	17	18	11	4

Padded input feature map



Kernel

1	1	1
1	1	1
1	1	1



Input feature map

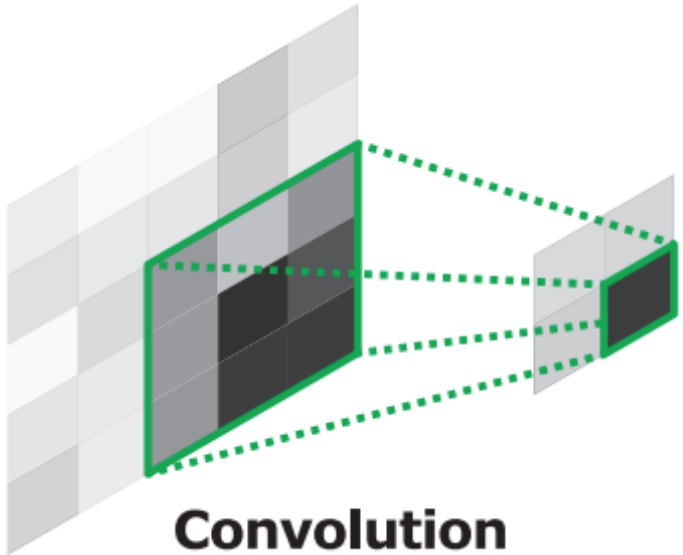
1	2
3	4

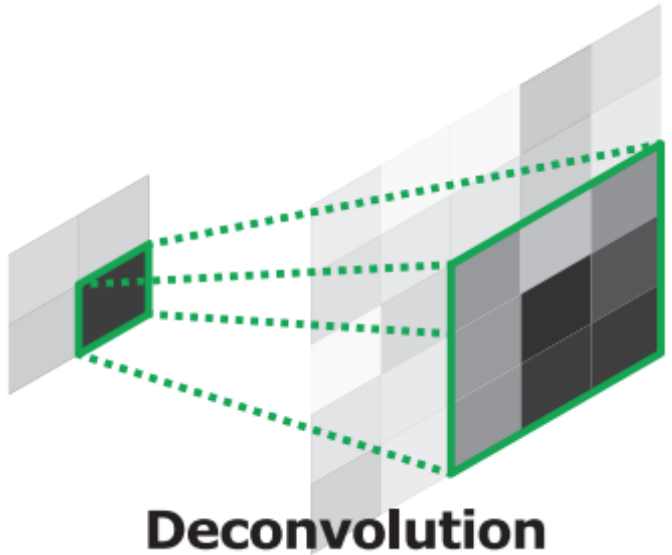
Cropped output feature map

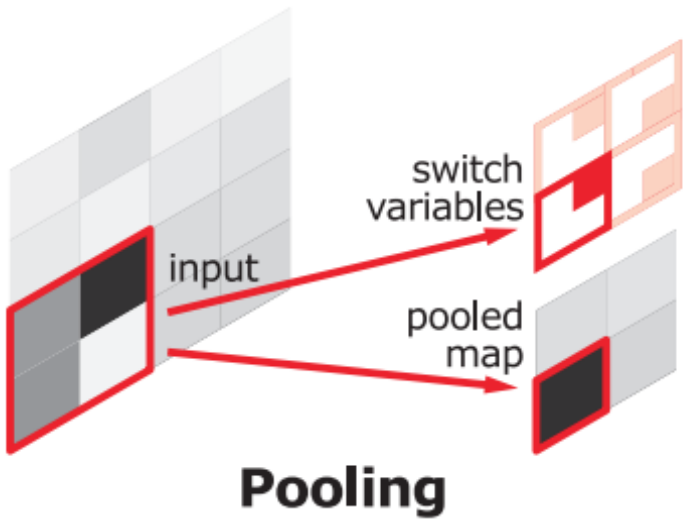
18	31	34	21
32	55	60	37
38	66	64	43
24	41	44	27

Padded input feature map

		1	2		
		3	4		







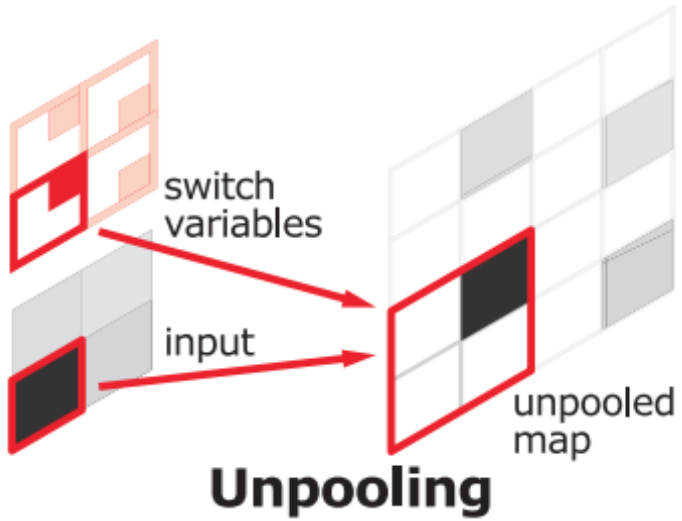


Image source: https://arxiv.org/pdf/1505.04366.pdf

Upsampling by “Unpooling”

Max Pooling
Remember which element was max!

1	2	6	3
3	5	2	1
1	2	2	1
7	3	4	8

Input: 4 x 4

5	6
7	8

Output: 2 x 2

Rest of the network

Max Unpooling
Use positions from pooling layer

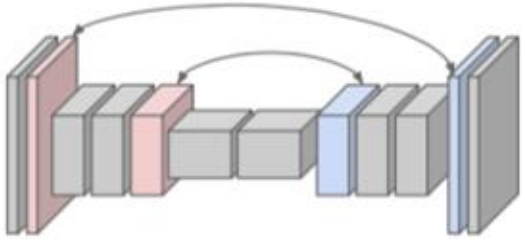
1	2
3	4

Input: 2 x 2

0	0	2	0
0	1	0	0
0	0	0	0
3	0	0	4

Output: 4 x 4

Corresponding pairs of downsampling and upsampling layers

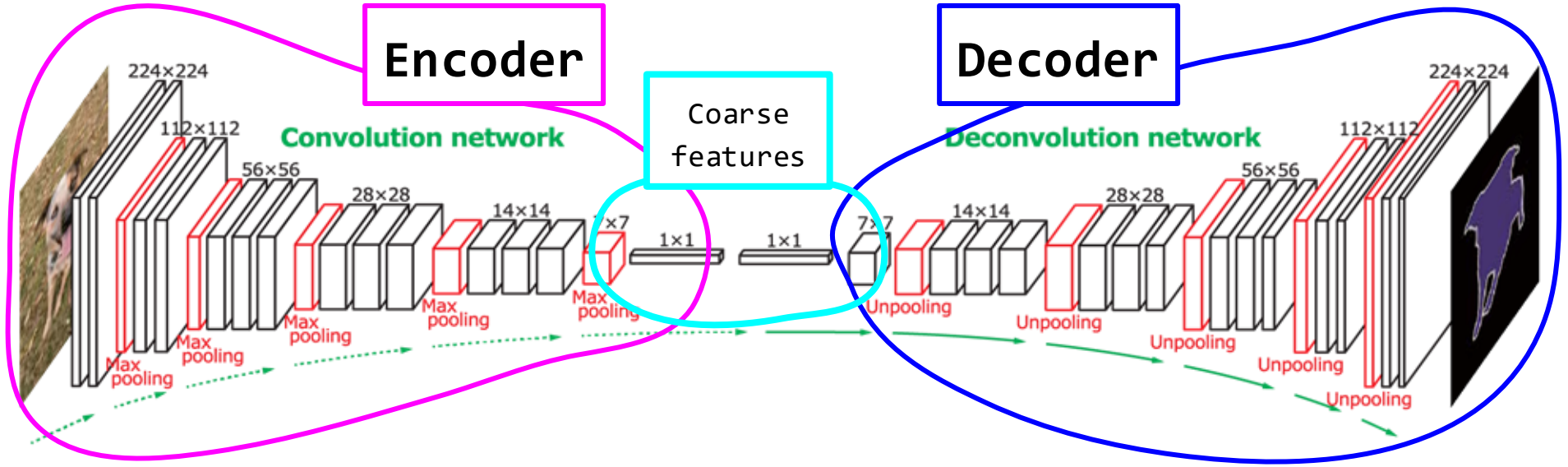


(switch variables)

The output of an unpooling layer is an **enlarged, yet sparse activation** map → deconvolution layers **densify** the sparse activations

Adapted from Fei-Fei Li et al.

‘Deconvolutional’ networks *learn to upsample*



Zeiler et al., Deconvolutional Networks, CVPR 2010
Long et al., Fully Convolutional Models for Semantic Segmentation, CVPR 2015
Noh et al., Learning Deconvolution Network for Semantic Segmentation, ICCV 2015

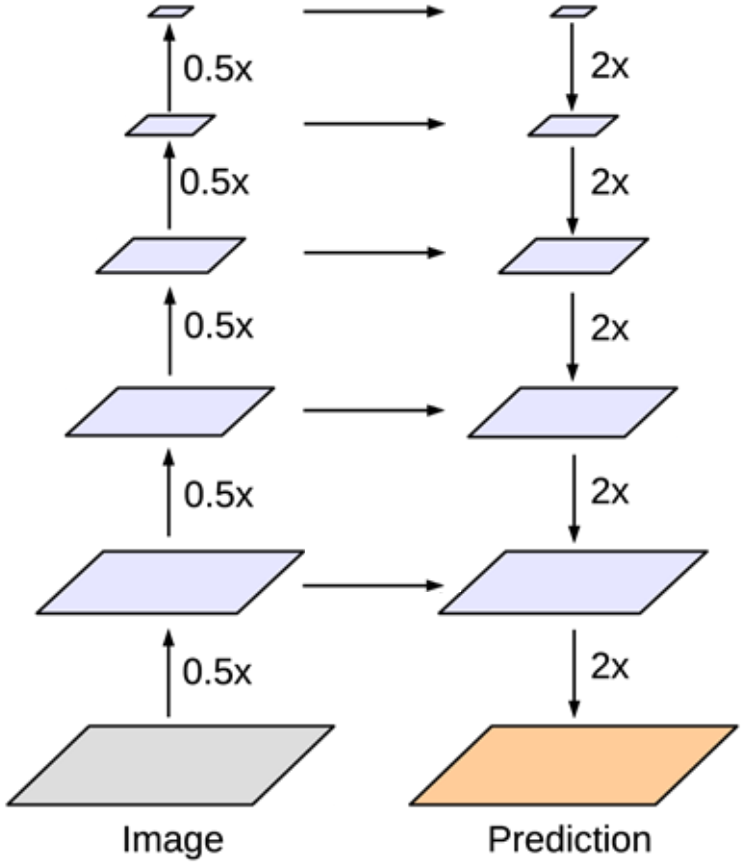
Image source: https://arxiv.org/pdf/1505.04366.pdf

FCN with Connections

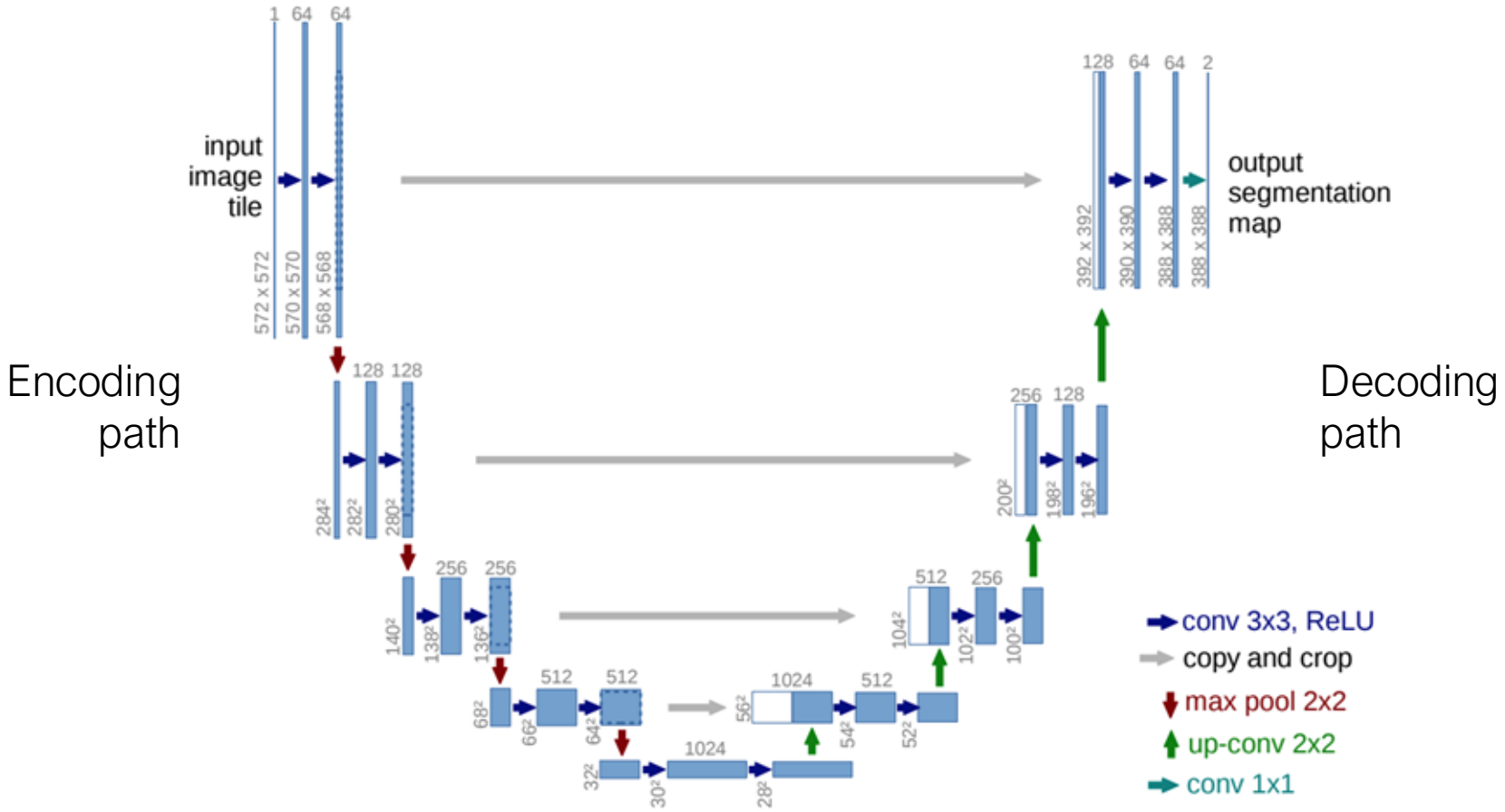
An alternative to the learnable upsampling, is to use **connections** between the encoder and the decoder

Allows to transfer fine-grain features from the encoder to the decoder

Examples: U-Net/Segnet



U-Net

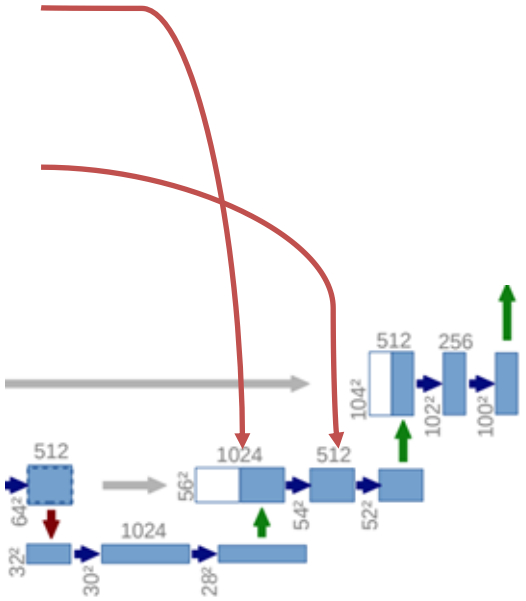


Ronneberger et al., U-net: Convolutional networks for biomedical image segmentation. MICCAI 2015
https://arxiv.org/abs/1505.04597

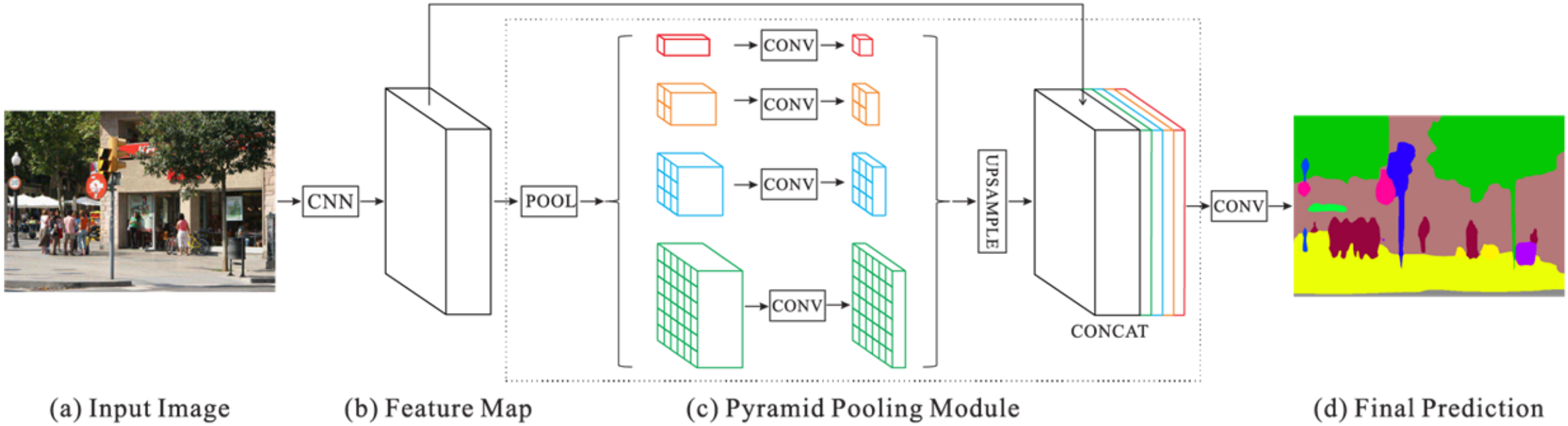
U-Net

Connections are needed to allow **recovering the high resolution** lost in the encoding path

- After upsampling, the symmetrical layer in the encoding path is **concatenated**
- The next convolution in the same level **recovers the fine detail** from the concatenated part

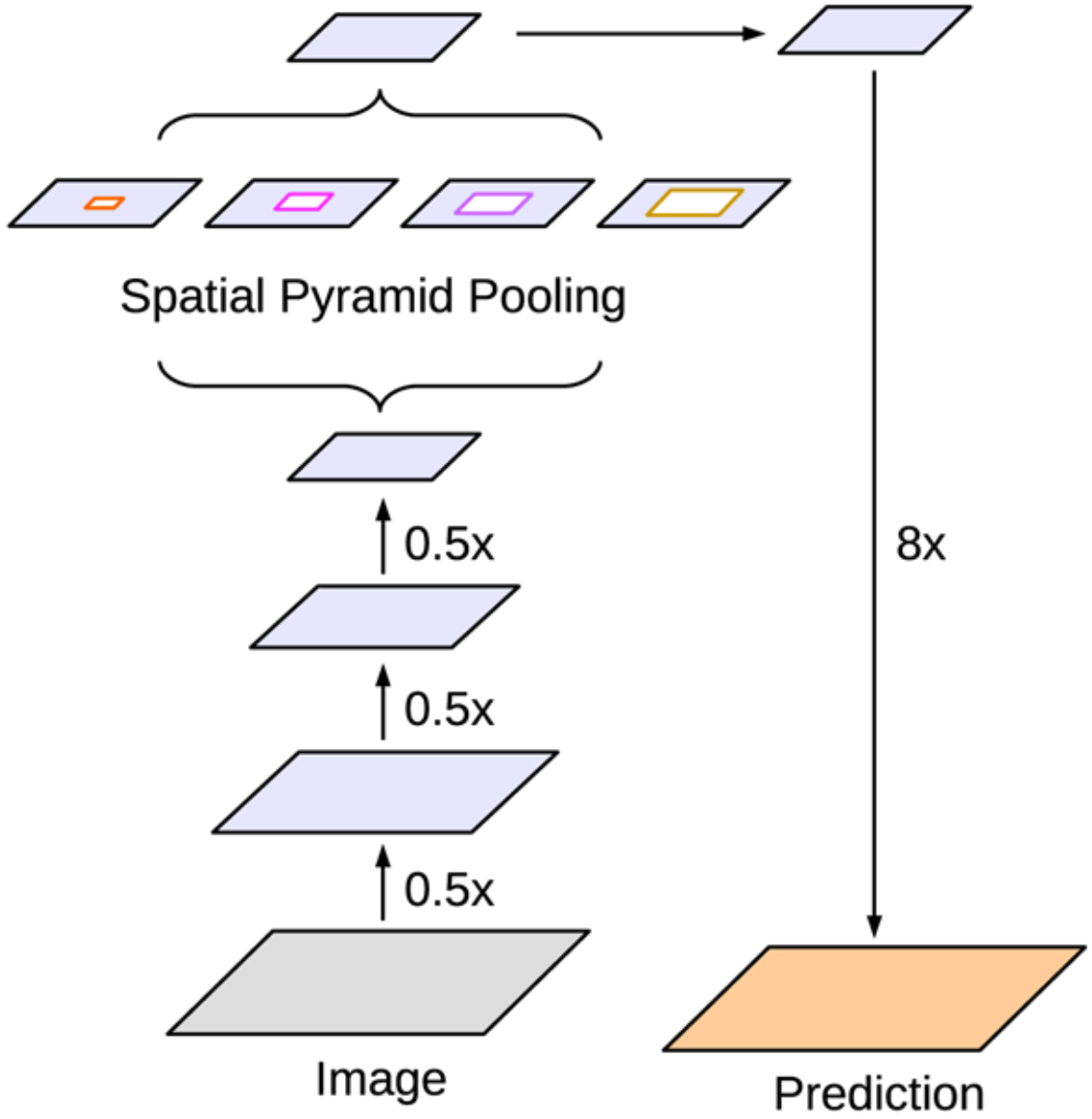


Spatial pyramid pooling



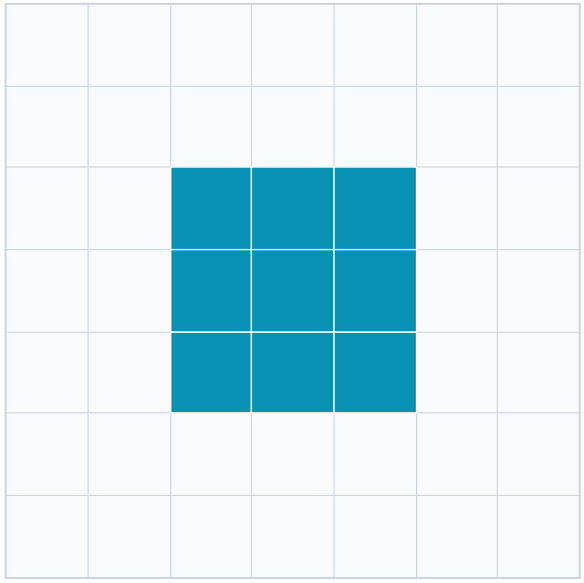
https://arxiv.org/pdf/1612.01105.pdf

Spatial pyramid pooling

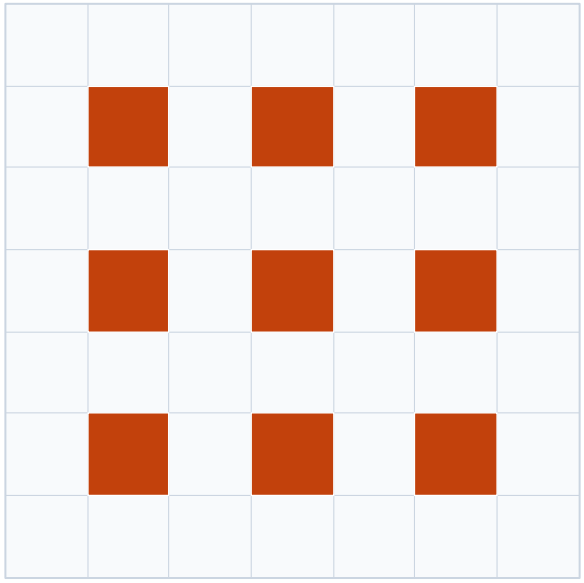


Dilated (atrous) convolution

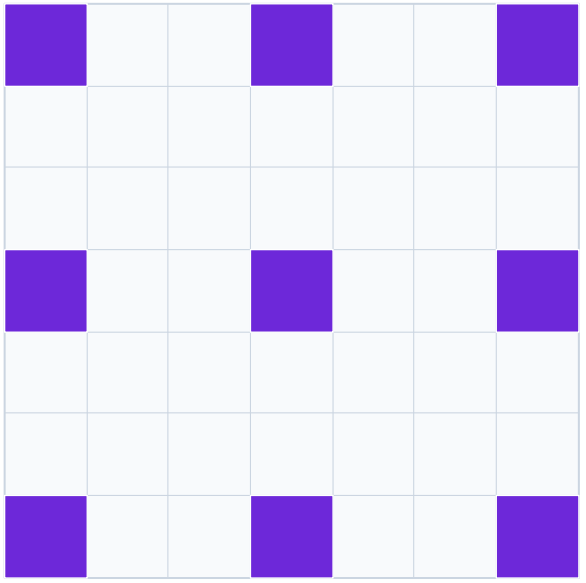
Expands the receptive field without losing resolution or adding parameters



rate = 1 (standard 3×3)



rate = 2 (3×3, 5 px receptive)

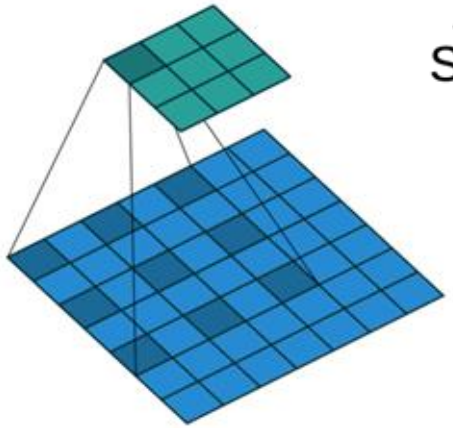


rate = 3 (3×3, 7 px receptive)

Yu & Koltun, Multi-Scale Context Aggregation by Dilated Convolutions, ICLR 2016

DeepLabv3+

Atrous convolutions
Spaced inputs



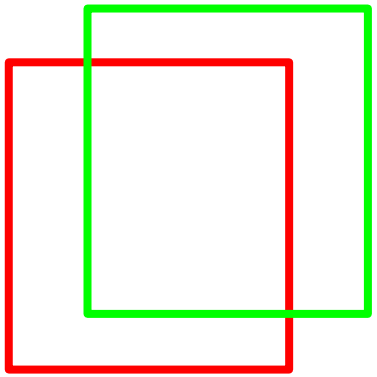
https://arxiv.org/pdf/1802.02611.pdf

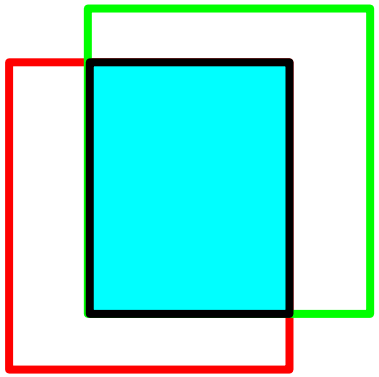
Scoring segmentation

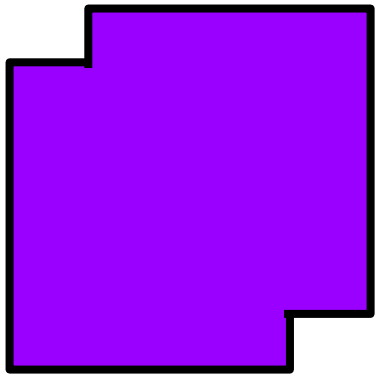
- **Pixel accuracy**
  - Not very useful, due to class imbalance (objects vs. background)
- **Jaccard index** or **Jaccard similarity coefficient**

$$J = IoU$$ Intersection over Union

Intersection: Ground truth ∩ prediction







Union: Ground truth ∪ prediction

Scoring segmentation

- Dice's coefficient or Dice similarity coefficient (DSC)

$$DSC = \frac{2 * intersection}{total}$$

$$DSC = \frac{2J}{1+J}$$

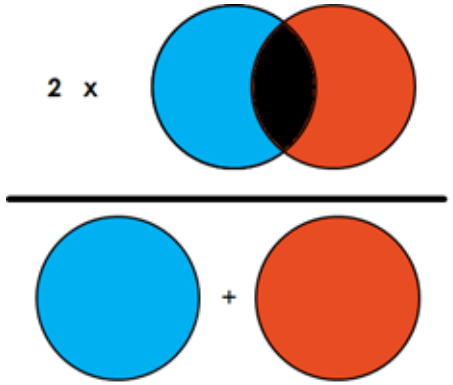


Image source: https://towardsdatascience.com/metrics-to-evaluate-your-semantic-segmentation-model-6bcb99639aa2

Loss functions for segmentation

Which loss to minimise during training?

Cross-entropy (per pixel)

$-\Sigma\, y_k \log(\hat{y}_k)$

Default choice. Classes roughly balanced.

Fails under heavy imbalance — background dominates the gradient.

Weighted / focal CE

$(1 - \hat{y})^{\gamma} \cdot CE$

Moderate imbalance, or hard-example mining.

Down-weights easy pixels (focal) or rescales per class.

Dice loss

$1 - 2 \cdot |A \cap B| \,/\, (|A| + |B|)$

Heavy imbalance. Small foreground, e.g. tumours, lesions, thin structures.

Directly optimises overlap. Unstable for very small / empty masks.

Tversky / focal Tversky

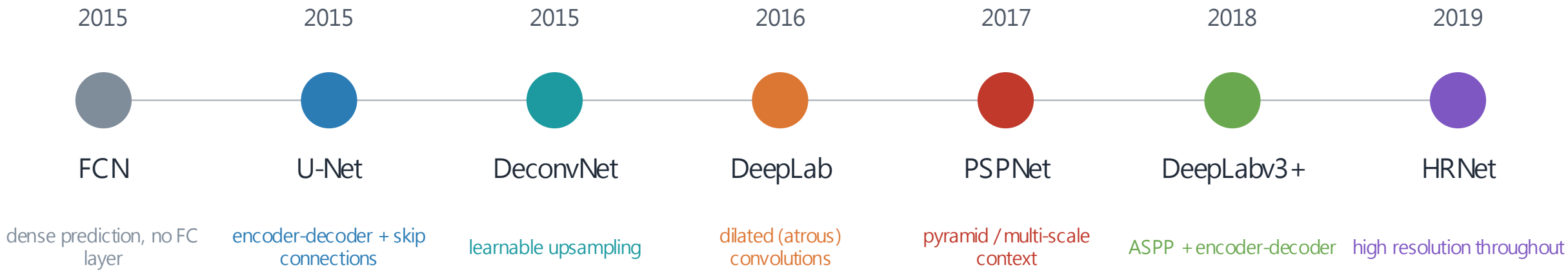
$1 - TP \,/\, (TP + \alpha \cdot FP + \beta \cdot FN)$

Imbalance + explicit trade-off between false positives and false negatives.

α, β are hyperparameters → β>α penalises missed foreground.

In medical imaging and other imbalanced settings, combining losses (e.g. CE + Dice) often trains more stably than either alone.

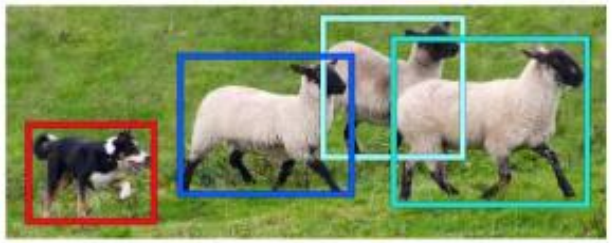
Semantic segmentation evolution



Each step finds a new way to bring more context into every pixel's prediction

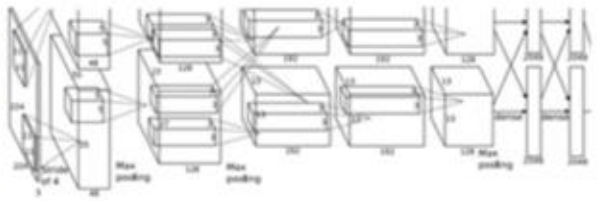
Object Detection

- What about object detection?
  - The goal is to predict a location (defined by a bounding box) of the objects



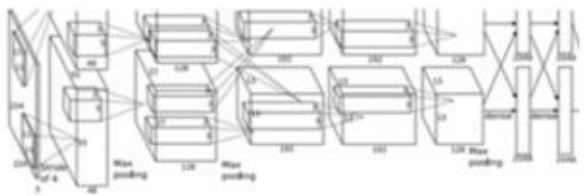
Object Detection as Regression?

Each image needs a different number of outputs



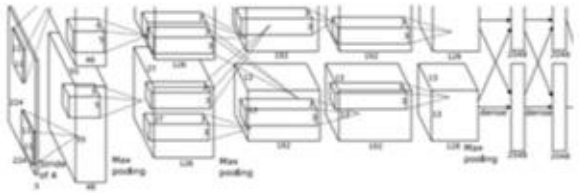
CAT: (x, y, w, h)

4 numbers



DOG: (x, y, w, h)
DOG: (x, y, w, h)
CAT: (x, y, w, h)

12 numbers

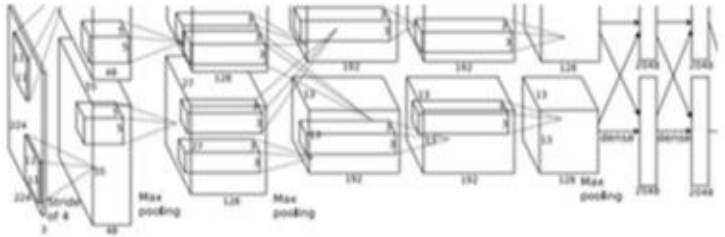


DUCK: (x, y, w, h)
DUCK: (x, y, w, h)
....

? numbers

Sliding Window

Apply a CNN to many different crops of the image, CNN classifies each crop as object or background

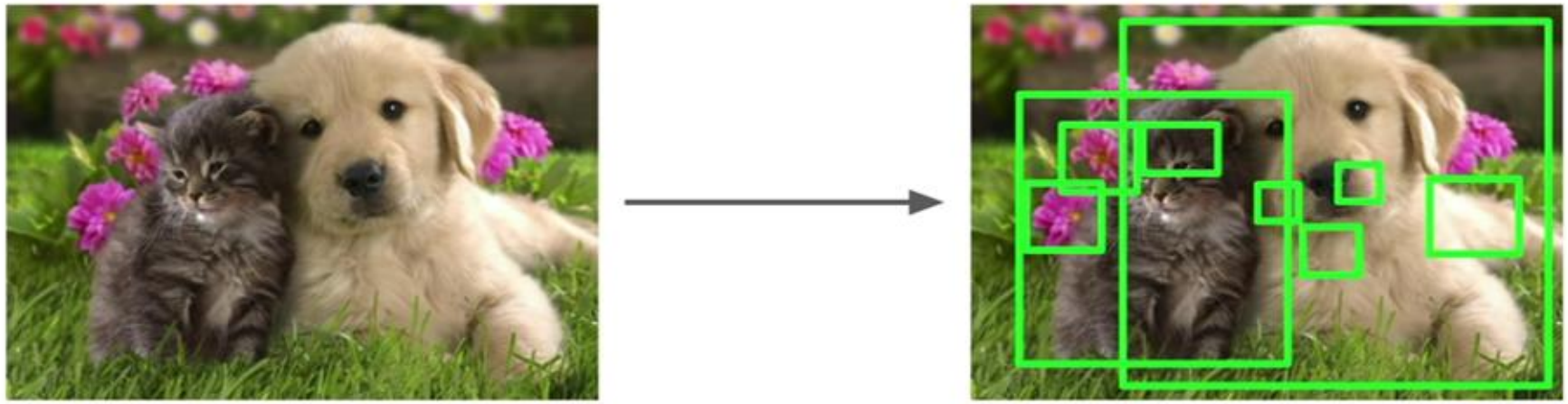


Dog? YES
Cat? NO
Background? NO

Problem: Need to apply CNN to huge number of locations and scales, very computationally expensive

Region Proposal

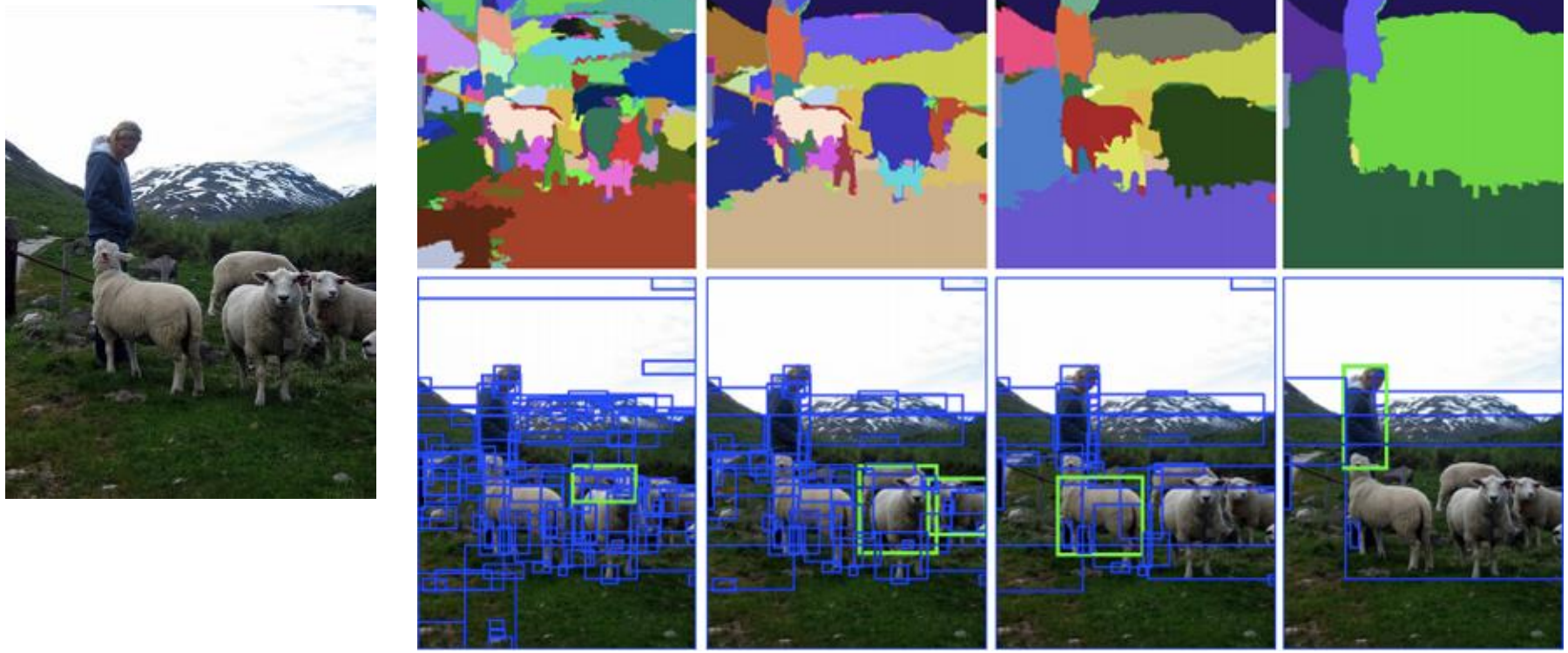
- Find image regions that are likely to contain objects, for example using **Selective Search**



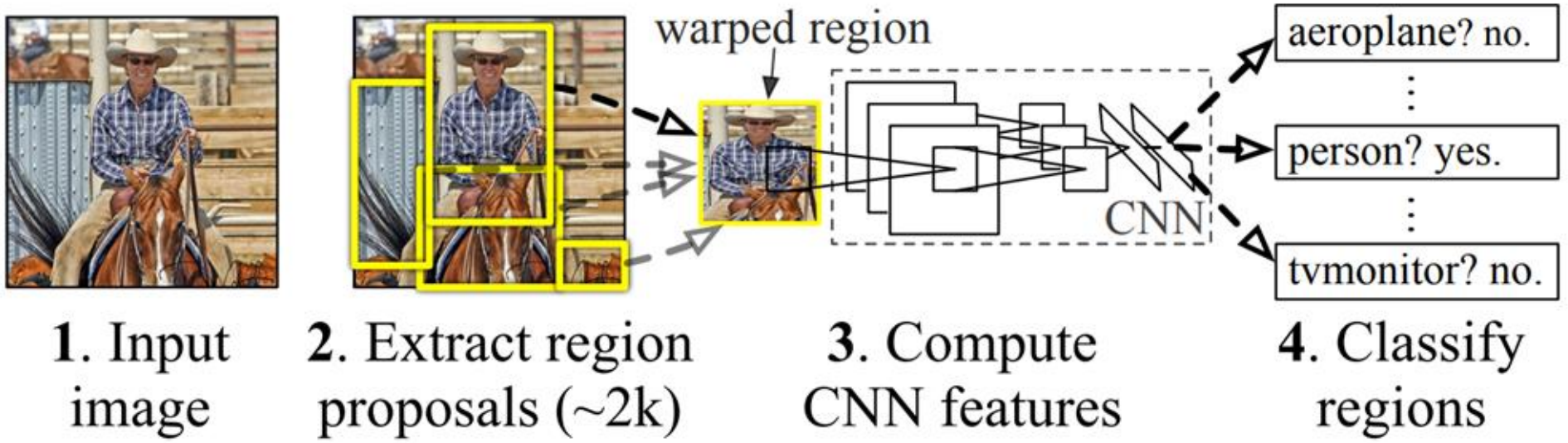
Alexe et al, "Measuring the objectness of image windows", TPAMI 2012
Uijlings et al, "Selective Search for Object Recognition", IJCV 2013
Cheng et al, "BING: Binarized normed gradients for objectness estimation at 300fps", CVPR 2014
Zitnick and Dollar, "Edge boxes: Locating object proposals from edges", ECCV 2014

Region Proposal

Regions defined by similarity on color, texture, ...



R-CNN: Region-based CNN

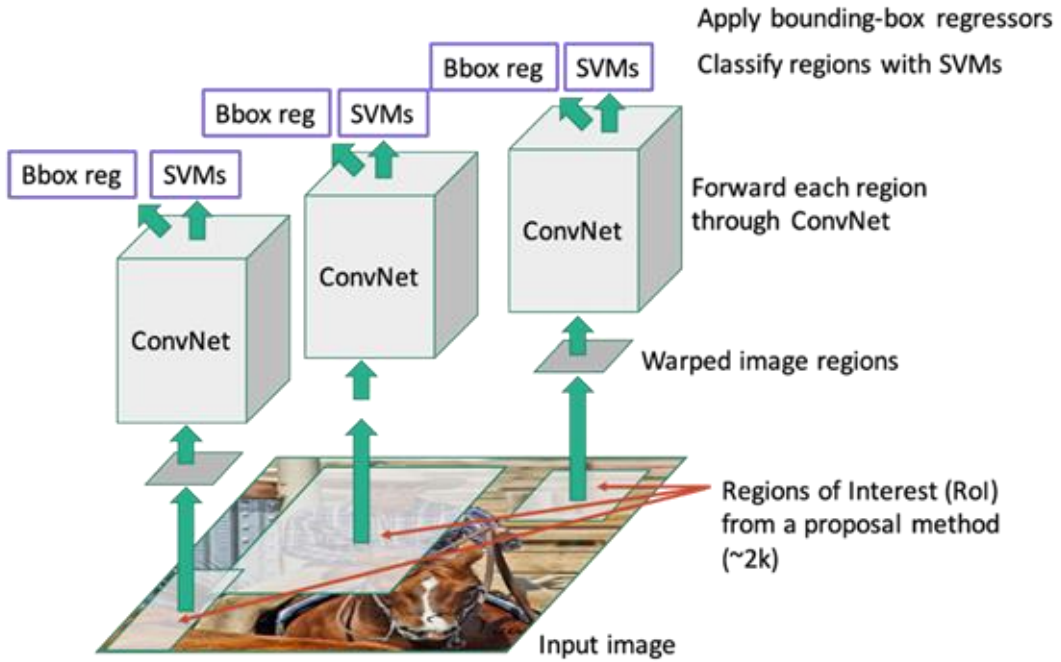


+bbox regression

Girshick et al.

R-CNN is slow

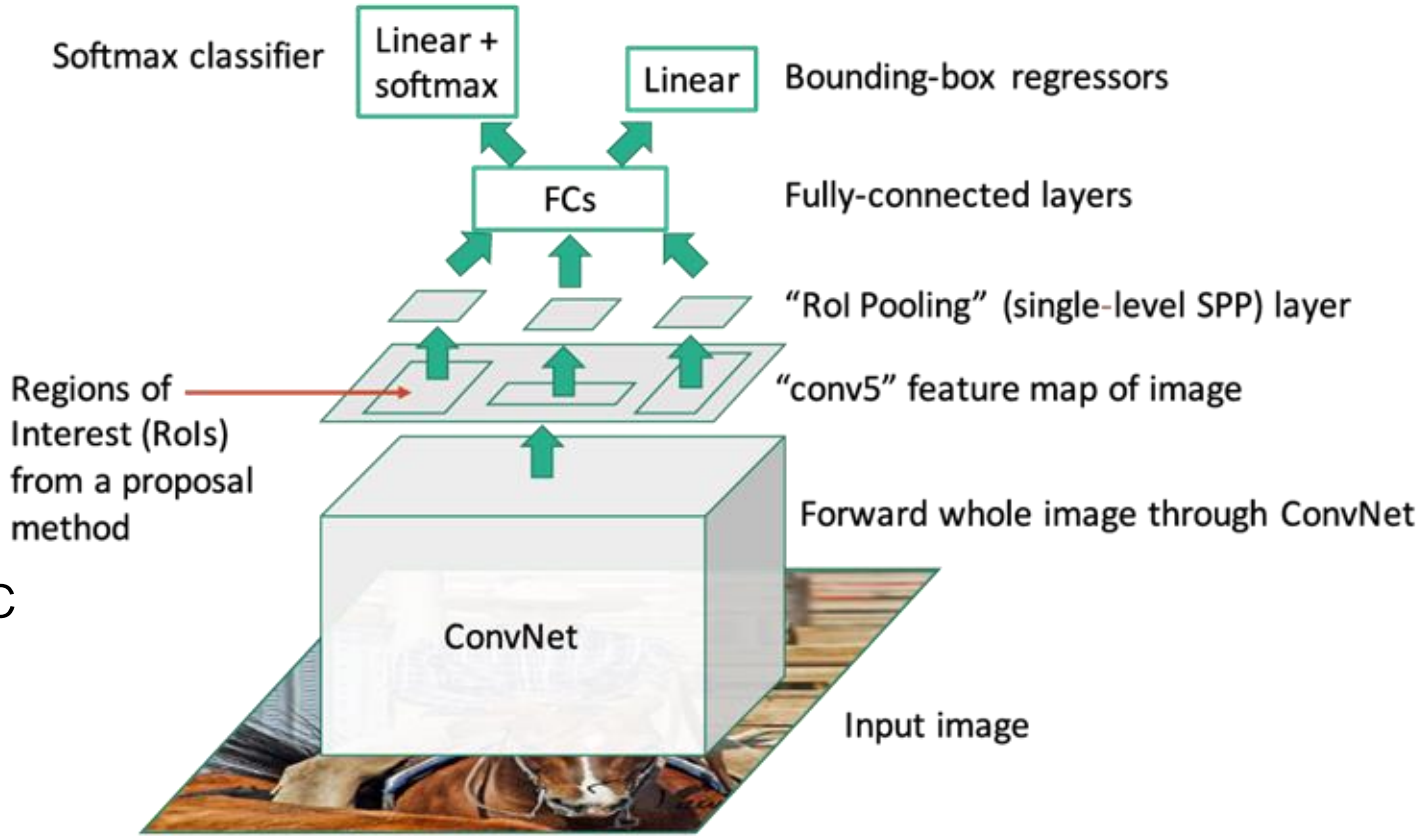
Run CNN independently over every ROI



adapted from Joseph Redmon

Fast R-CNN

- Run CNN once, extract features using ROI pooling
- ROI Pool:
  Convert variable sized
  ROI to fixed size output
- Much faster, no independent network evals (except last linear layer)
- Still slow region proposer, selective search takes ~2 sec



adapted from Joseph Redmon

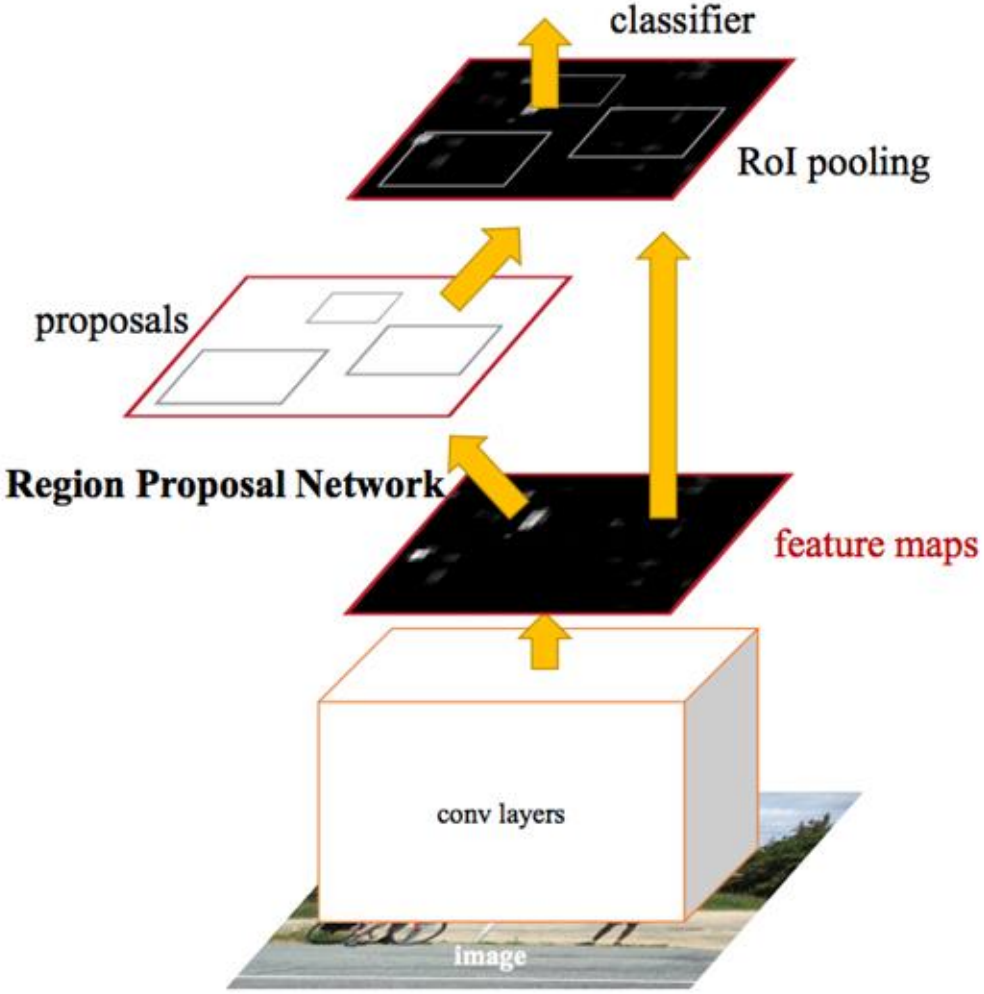
Faster R-CNN

Uses CNN to propose regions and generate features → **Region Proposal Network**

ROI Pool to fix size of ROI features

Additional layers to classify and predict bbox for ROIs

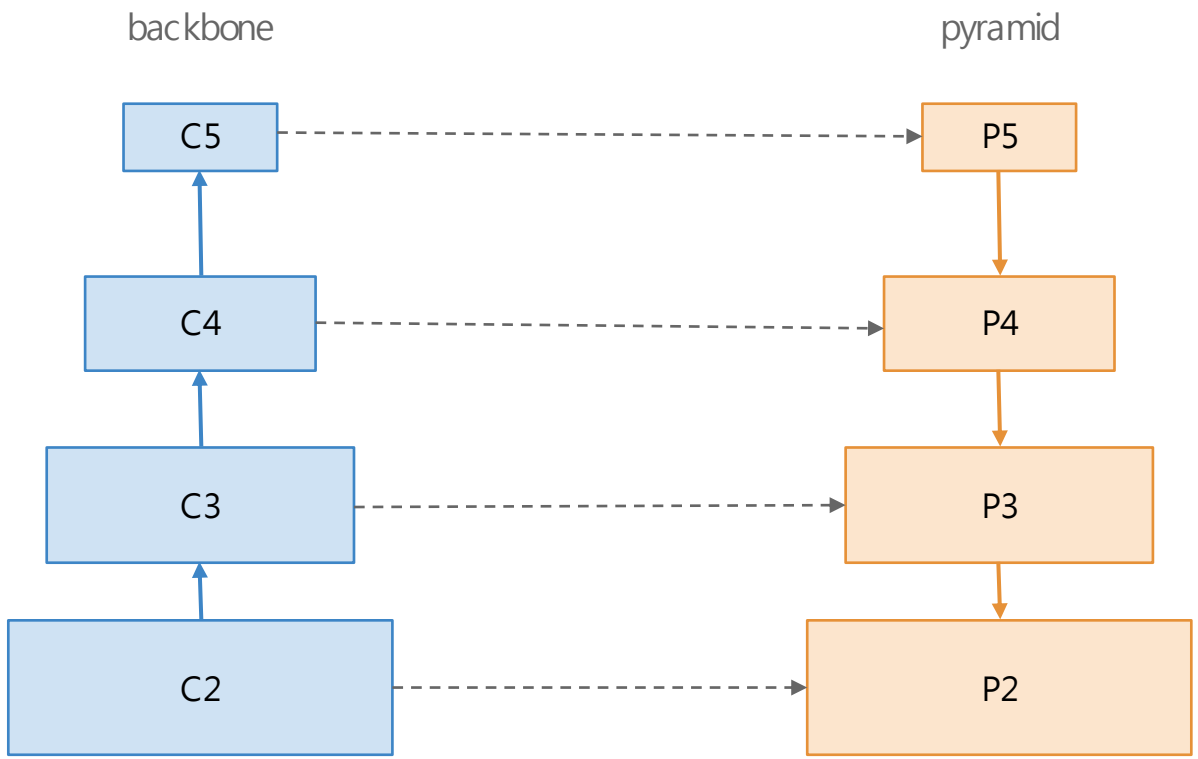
End to end object detection.



Ren et al. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks, 2016
https://arxiv.org/abs/1506.01497

Feature Pyramid Networks (FPN)

Objects have different scales. From which feature level should we detect each one?



Predict only from the last (deepest) feature map.

✗ C5 is semantically strong but spatially coarse.

✗ Small objects disappear before we reach it.

Build a Feature Pyramid and predict at every level:

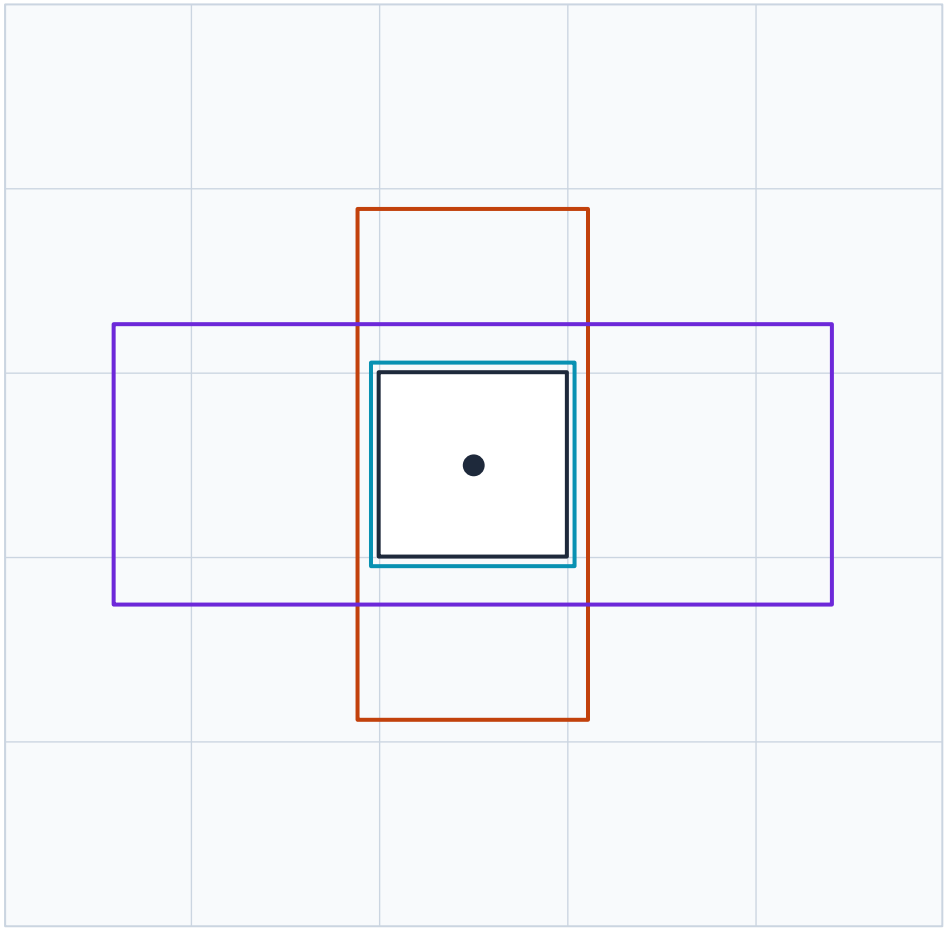
- small objects → P2
- large objects → P5

Same encoder-decoder + skip architecture as U-Net

Lin et al., Feature Pyramid Networks for Object Detection, CVPR 2017 · https://arxiv.org/abs/1612.03144

Anchors

Now that each pyramid level has good features, we need a way to propose boxes at each location.



one grid cell → k anchors

Anchors are fixed reference boxes at a location of the feature map. Different scales and aspect ratios cover the variety of object shapes.

For each anchor, the network predicts:

- objectness score — is there an object?
- 4 offsets (Δx, Δy, Δw, Δh) — refine the anchor
- class probabilities (in 1-stage detectors)

Faster R-CNN uses ~9 anchors/cell (3 scales × 3 ratios).

Ren et al., Faster R-CNN (2015) · Redmon & Farhadi, YOLO9000 (2017)

Anchor-free detectors

Anchors add hyperparameters (scales, ratios, IoU thresholds) and a **mismatch between the grid the grid and the objects**. Anchor-free methods let each feature-map location predict a box directly.

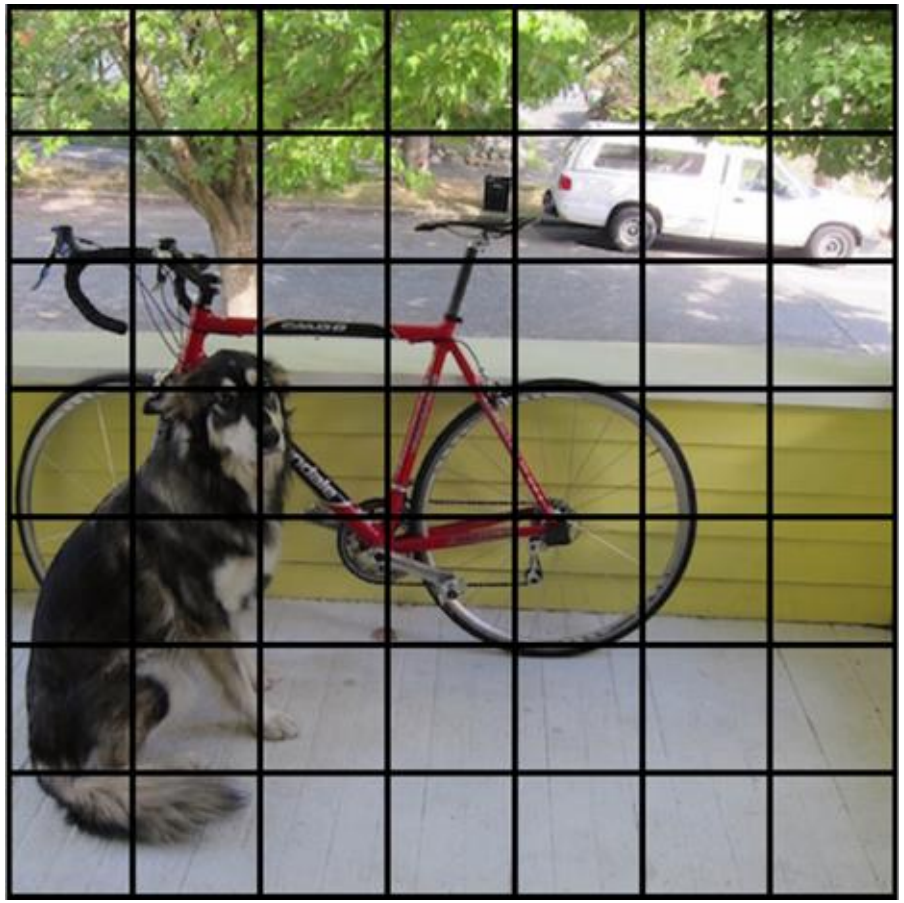
FCOS every pixel inside an object regresses 4 distances to the box edges *Tian et al., ICCV 2019*

CenterNet detects an object's centre as a peak in a heatmap, then regress its size *Zhou et al., 2019*

CornerNet predicts top-left and bottom-right corners as keypoints and pair them up *Law & Deng, ECCV 2018*

YOLO (You Only Look Once)

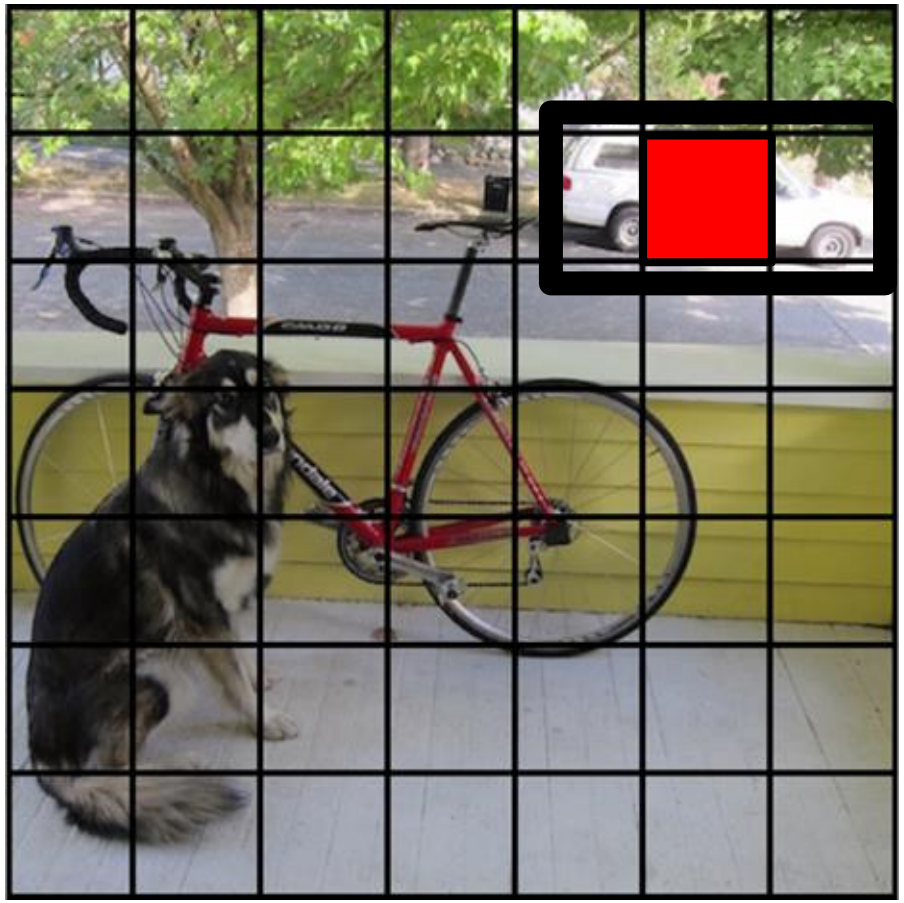
Split image into a grid



Ali Farhadi, Joseph Redmon

YOLO

Each cell predicts B bboxes(x,y,w,h) and confidences of each box: P(Object)



Ali Farhadi, Joseph Redmon

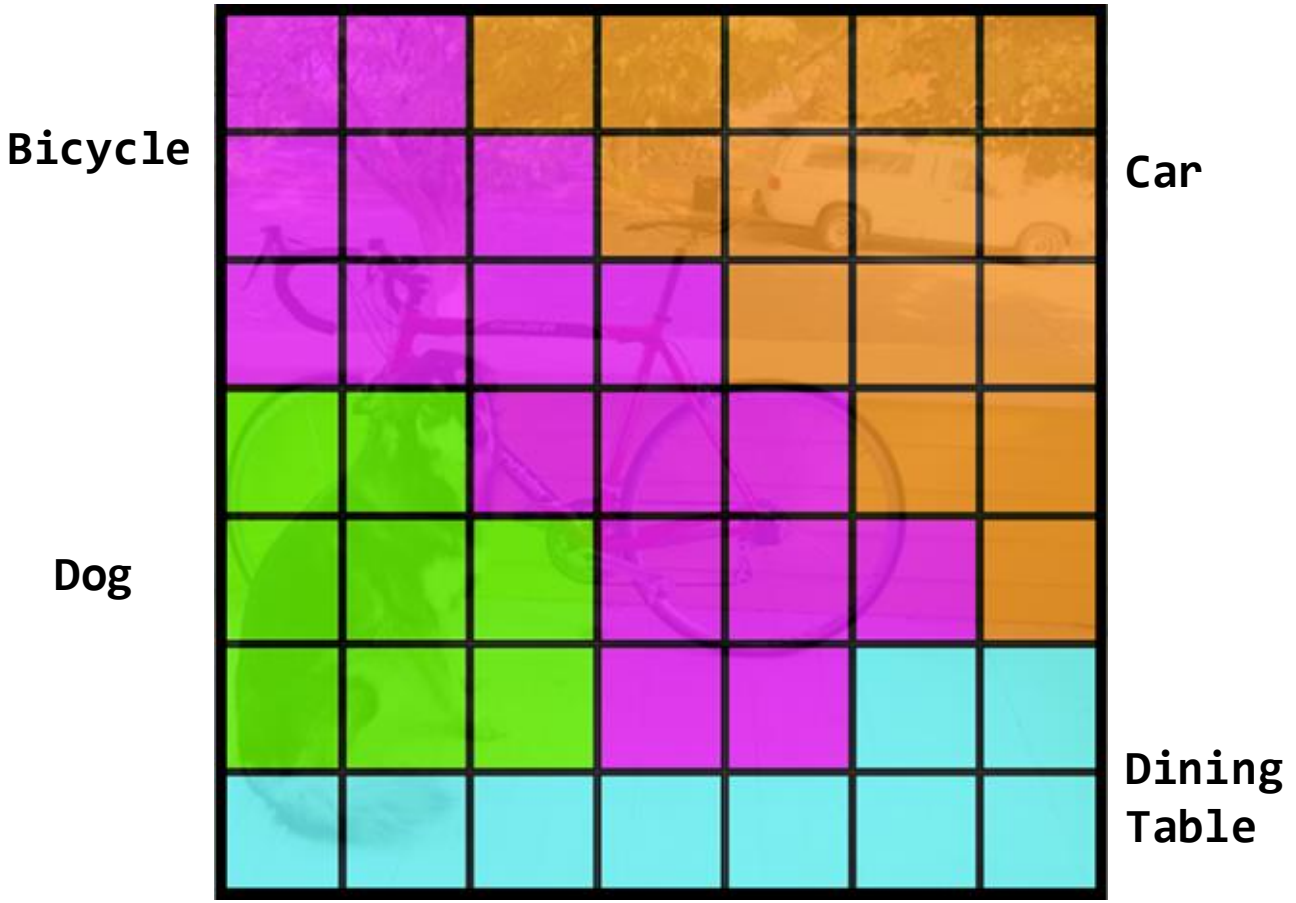
YOLO

Each cell predicts B bboxes(x,y,w,h) and confidences of each box: P(Object)

Ali Farhadi, Joseph Redmon

YOLO

Each cell also predicts a class probability

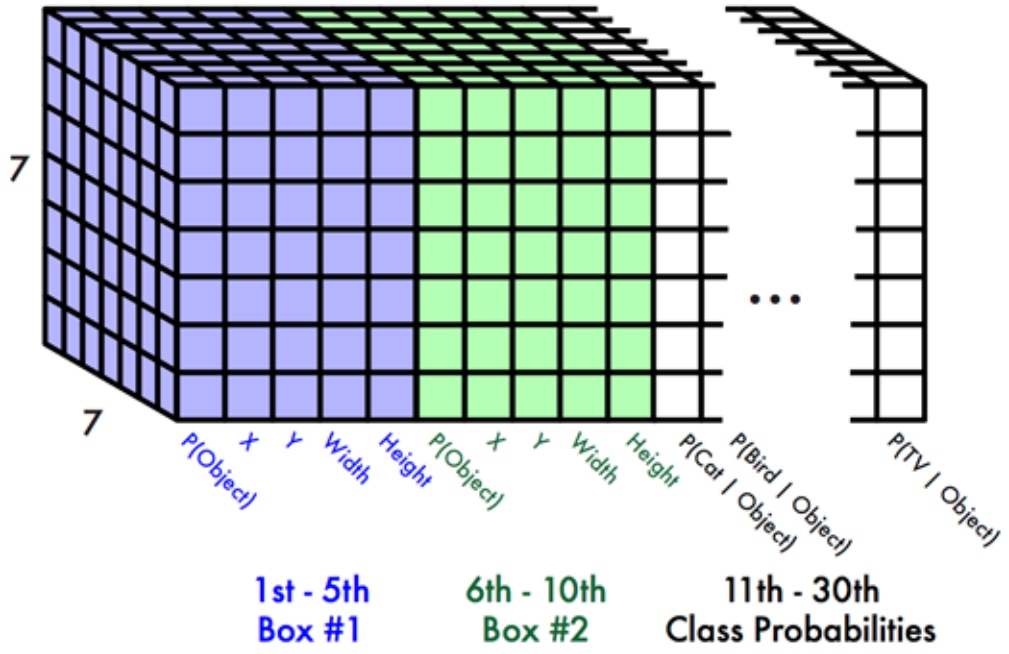


Ali Farhadi, Joseph Redmon

YOLO

Each cell predicts:

- For each bounding box:
  - 4 coordinates (x, y, w, h)
  - 1 confidence value
- Some number of class probabilities



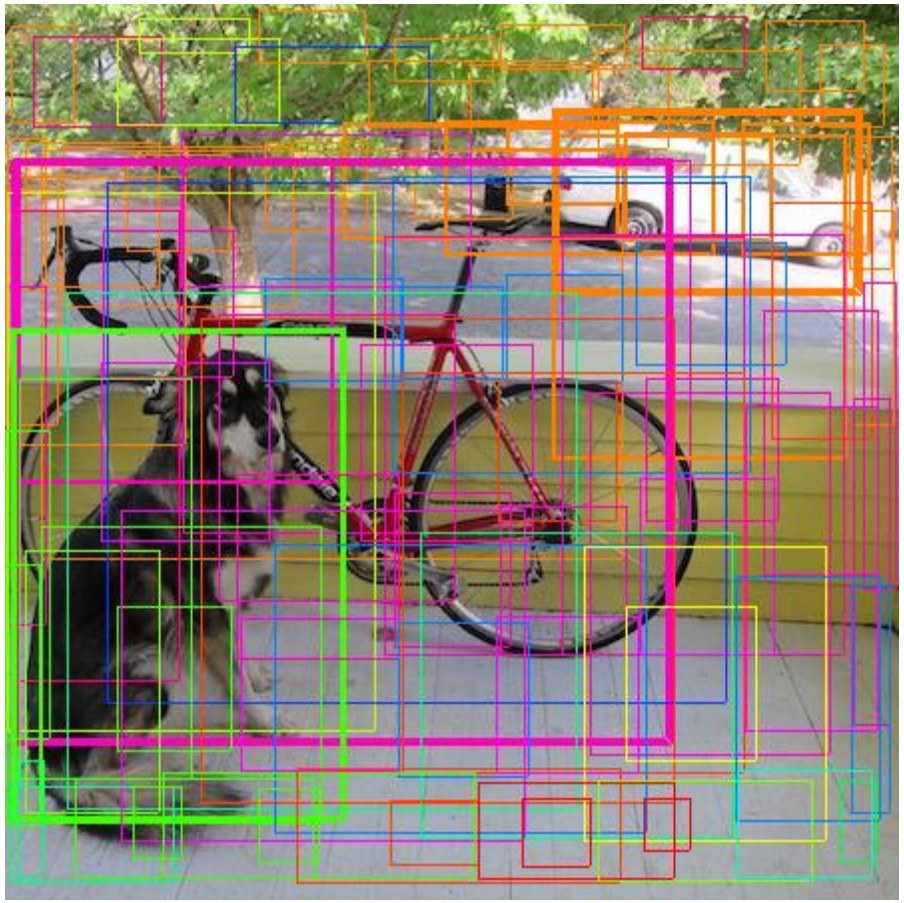
For Pascal VOC dataset:

- 7x7 grid
- 2 bounding boxes / cell
- 20 classes

7 x 7 x (2 x 5 + 20) = 7 x 7 x 30 tensor = **1470 outputs**

YOLO

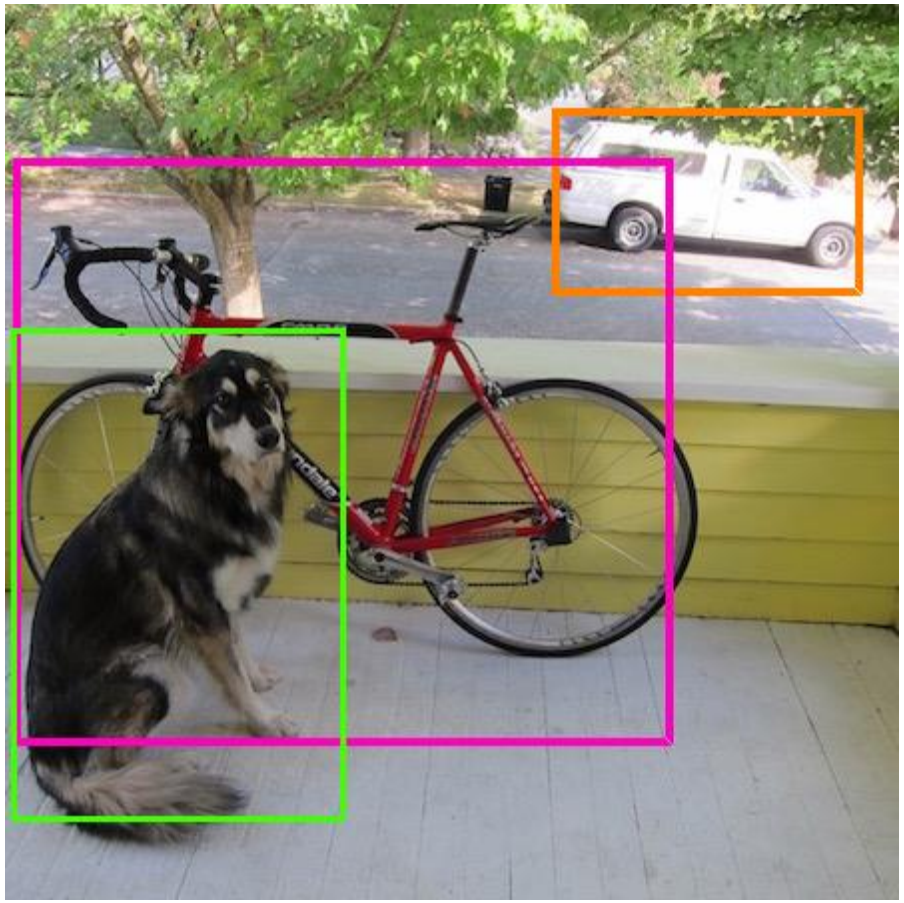
Then combine the bboxes and class predictions



Ali Farhadi, Joseph Redmon

YOLO

Finally, do threshold and non-maximum suppression

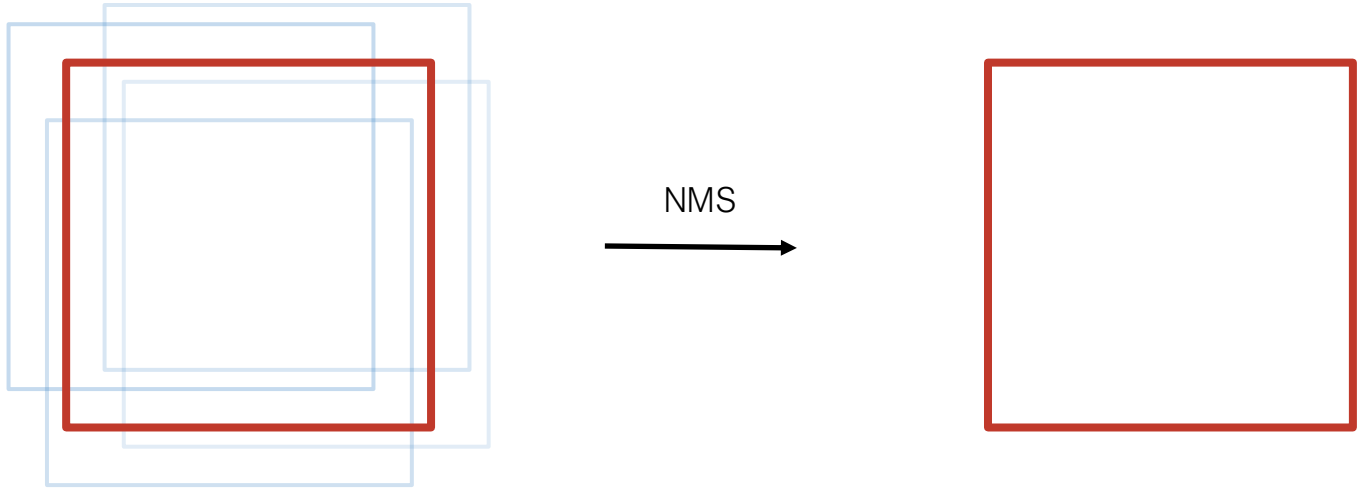


Ali Farhadi, Joseph Redmon

Non-Maximum Suppression (NMS)

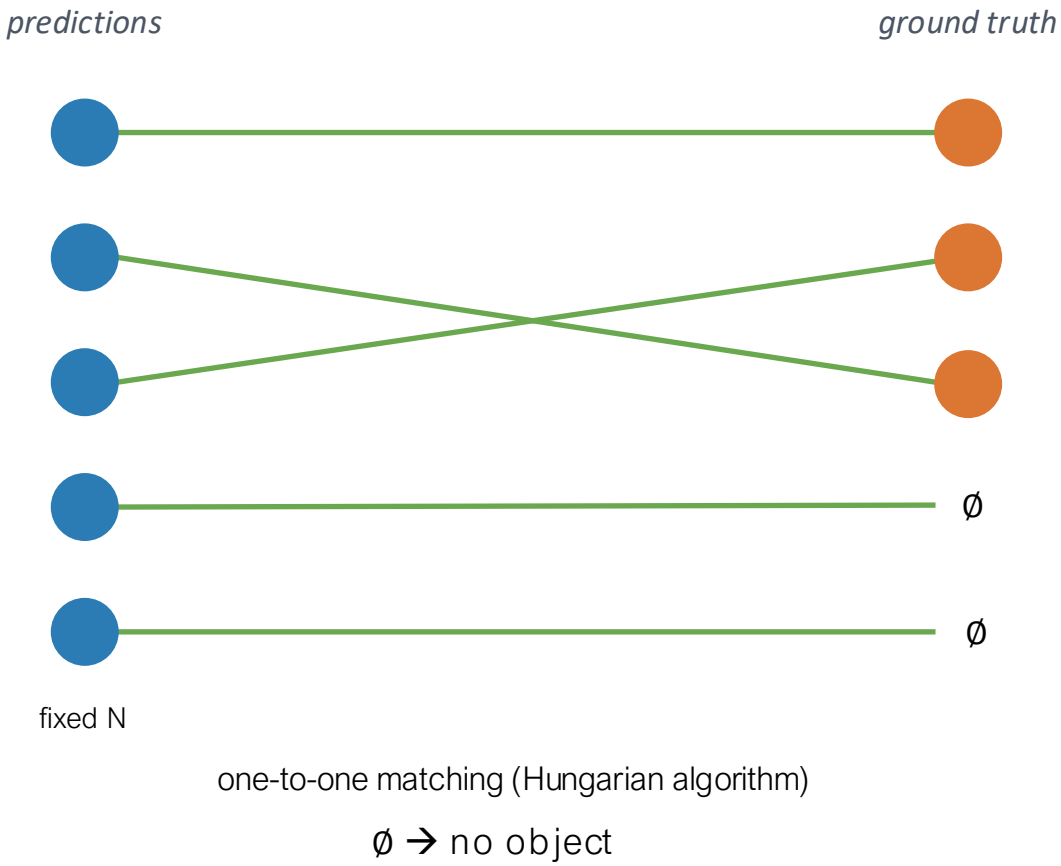
Detectors produce many overlapping boxes per object. NMS keeps the best one.

1. Sort boxes by confidence
2. Keep the top box
3. Remove overlapping boxes (IoU > 0.5)
4. Repeat, per class



Non-NMS approaches: how?

What if the model just learned to output one box per object?



Key idea

Match each prediction to **at most one** GT during training.

In other words, only one prediction can claim each real object. The rest are matched to ø and trained to output "no object".

Duplicates are penalised → no NMS at inference.

Architecture-agnostic

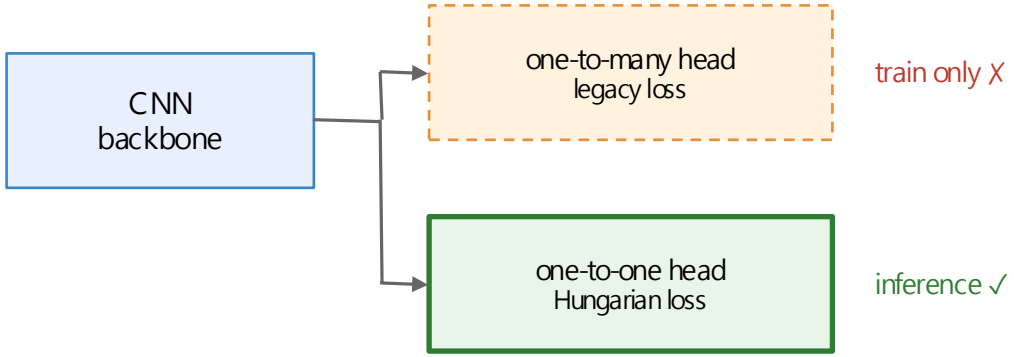
- YOLOv10 / YOLO26 — pure CNN
- DETR — transformer (2020)

Carion et al., DETR, ECCV 2020 · Wang et al., YOLOv10, NeurIPS 2024

Non-NMS approaches: how?

YOLOv10 / YOLO26 — dual-head

pure CNN · two heads at train time, one at inference

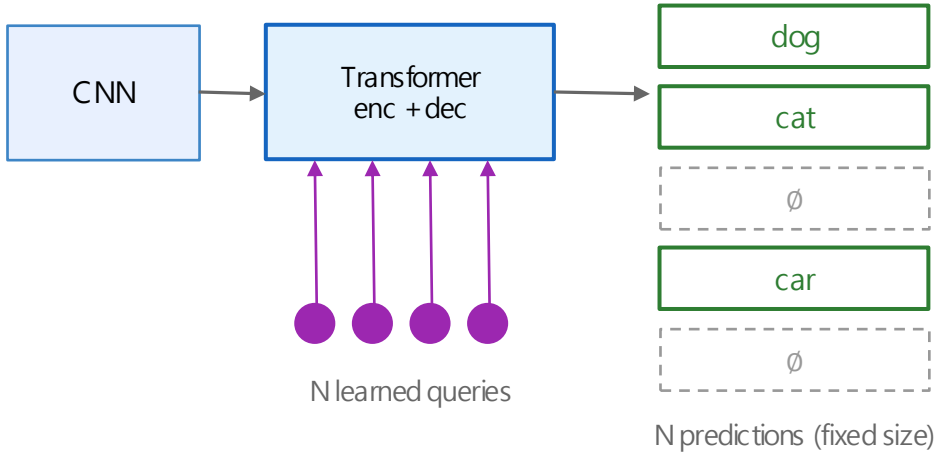


Why two heads?

one-to-one matching alone produces sparse supervision → slow, unstable training. The one-to-many head keeps training rich; the one-to-one head learns to pick the winner.

DETR — object queries

transformer · N learned queries, each predicts one object (or ∅)



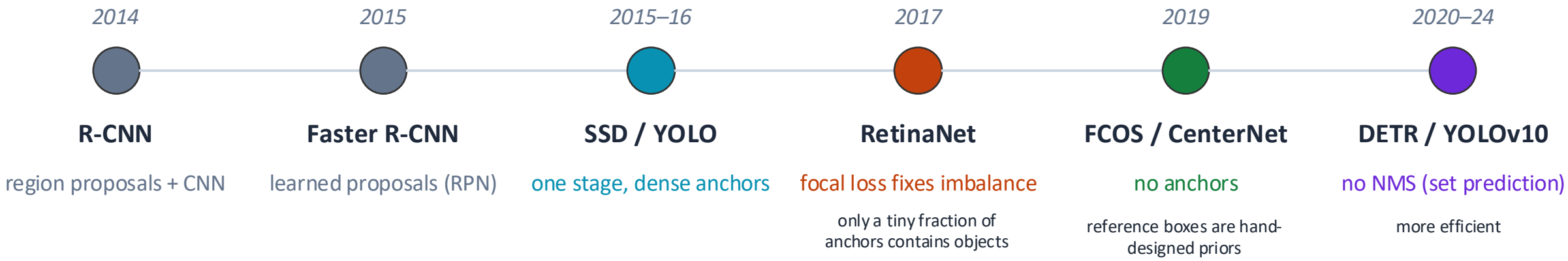
Why no duplicates?

Each query attends to the whole image and to the other queries, so they can specialise. Bipartite (Hungarian) matching at train time teaches each query to claim at most one object.

Both rely on Hungarian matching during training — the NMS-free property is a property of the loss, not the architecture.

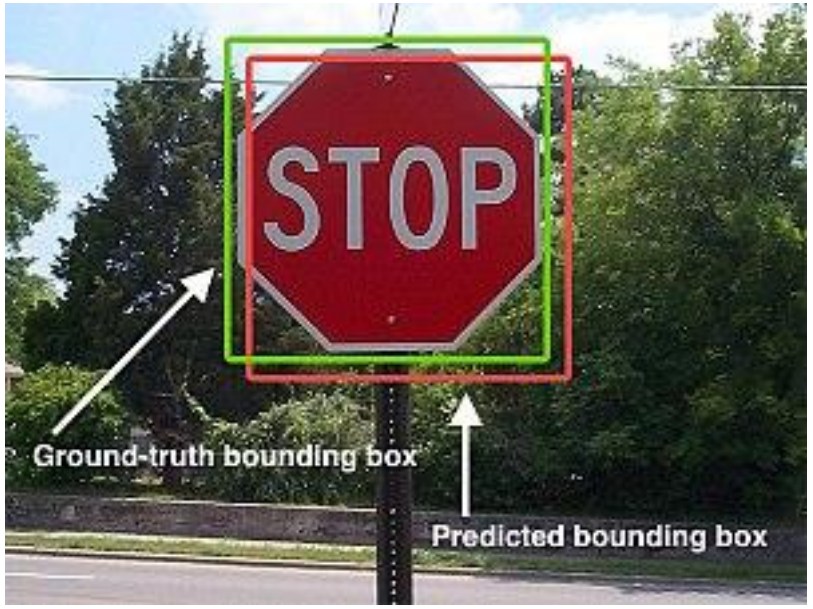
Carion et al., DETR, ECCV 2020 · Wang et al., YOLOv10, NeurIPS 2024 · Ultralytics YOLO26

Object detection evolution



Each step removes a hand-designed piece from the pipeline

Scoring object detection



Correct object detection?

A measure of similarity is needed. If this measure is larger than a **threshold**, we consider it correct

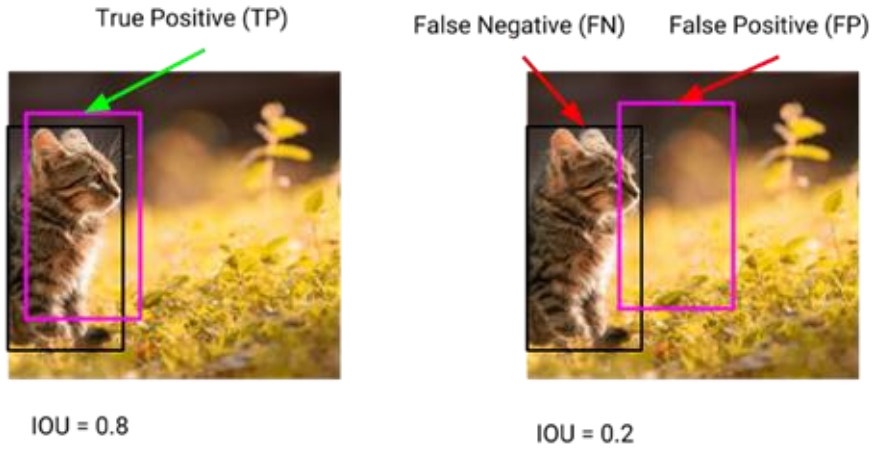
Image source: https://commons.wikimedia.org/wiki/File:Intersection_over_Union_-_object_detection_bounding_boxes.jpg

Scoring object detection

"Correct" bounding box:

IoU = Intersection / Union > 0.5

(or a different threshold)

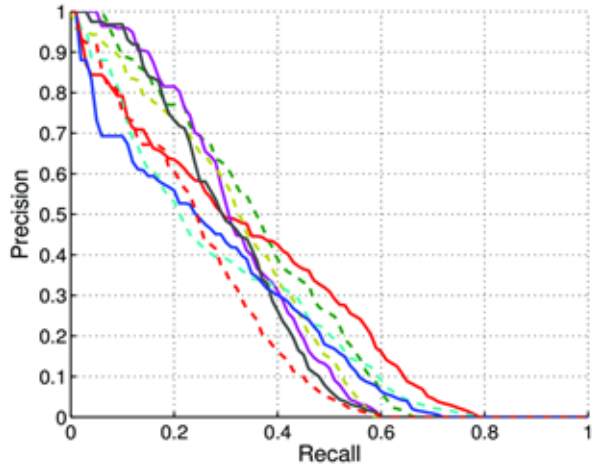


Recall: Correct bounding boxes / total ground-truth boxes

Precision: Correct bounding boxes / total predicted boxes

Typically, mean average precision (mAP) is used

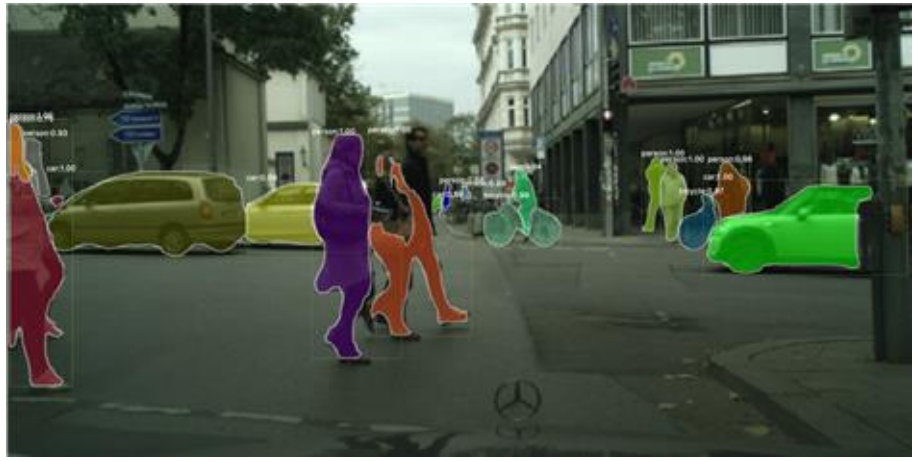
- AP is the area under the PR curve for different thresholds (only for a single class)
- mAP is the average AP over the different classes

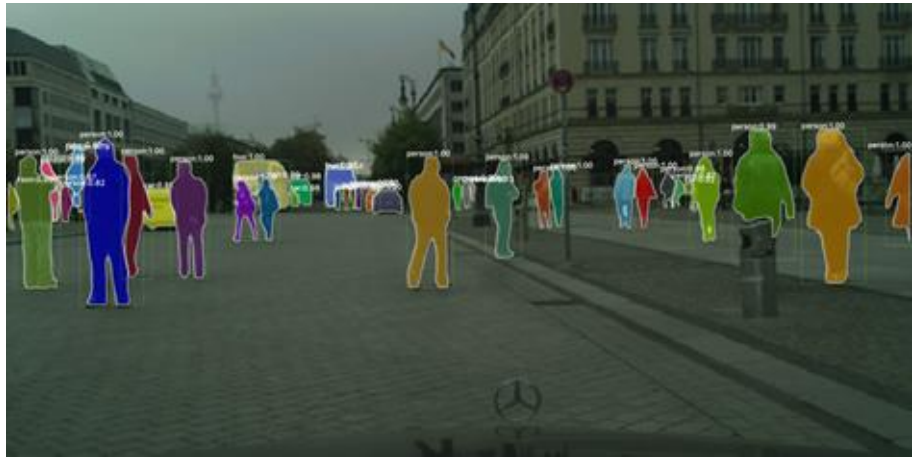


adapted from Ali Farhadi, Joseph Redmon

Instance Segmentation

Given an image produce instance-level segmentation
Which class does each pixel belong to
Also which instance

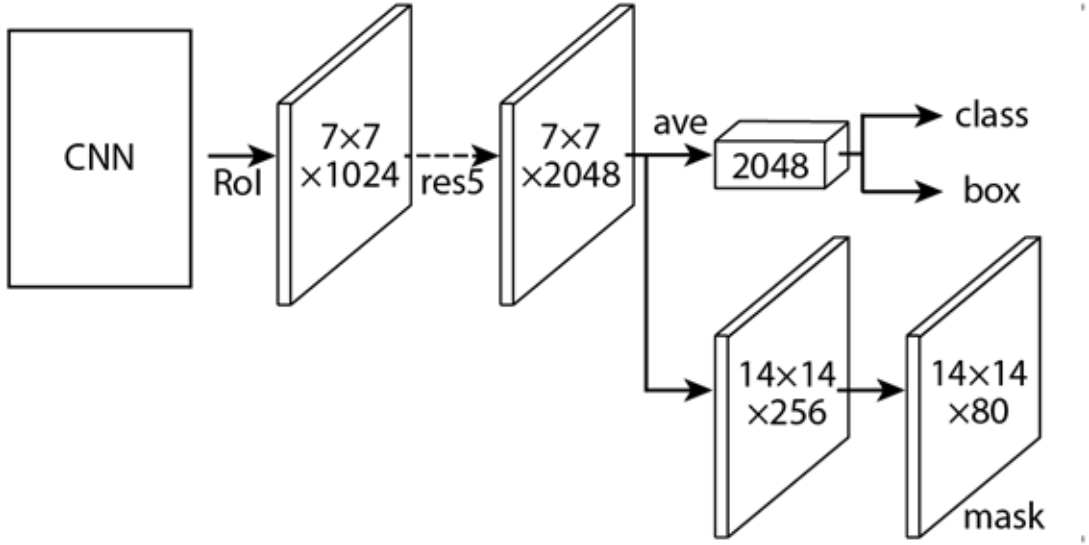




Ali Farhadi, Joseph Redmon

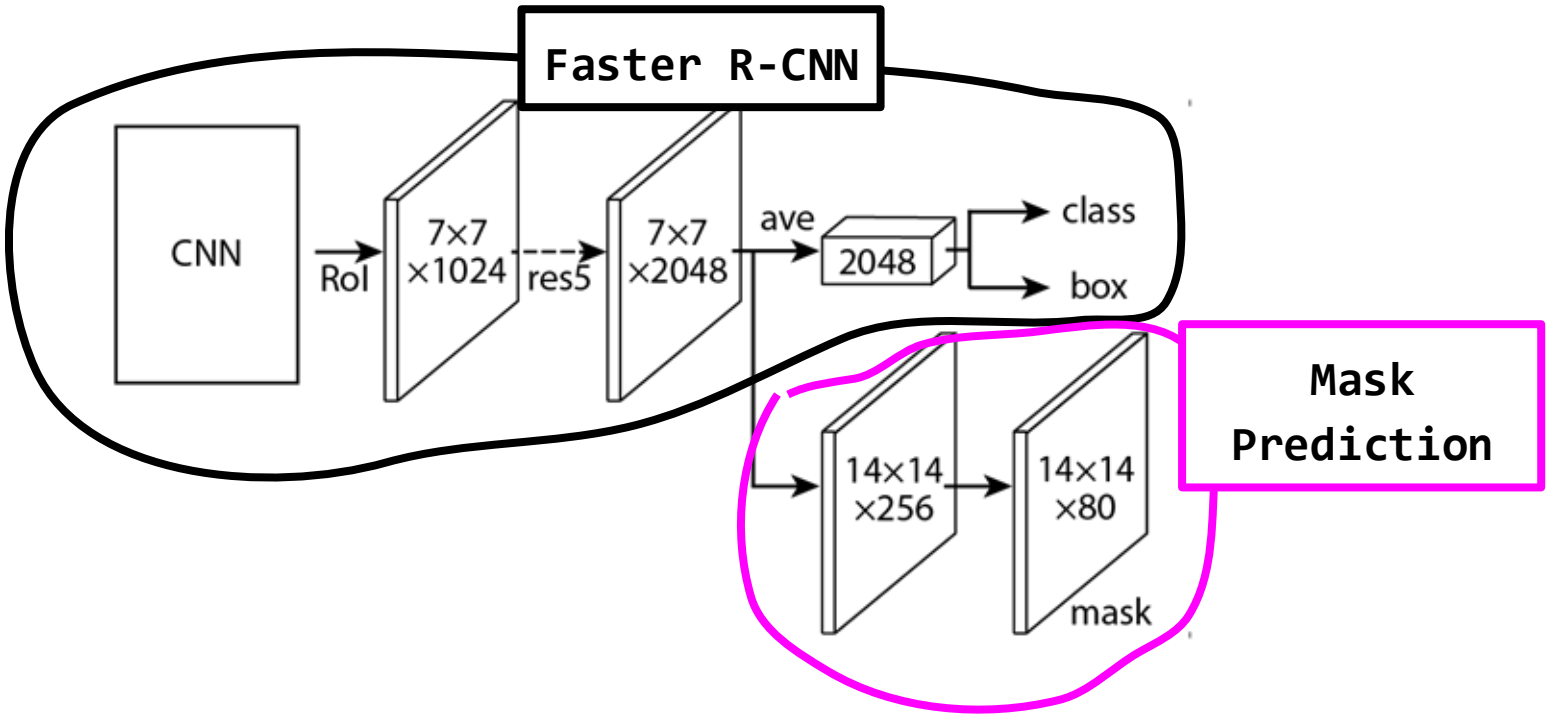
Mask R-CNN

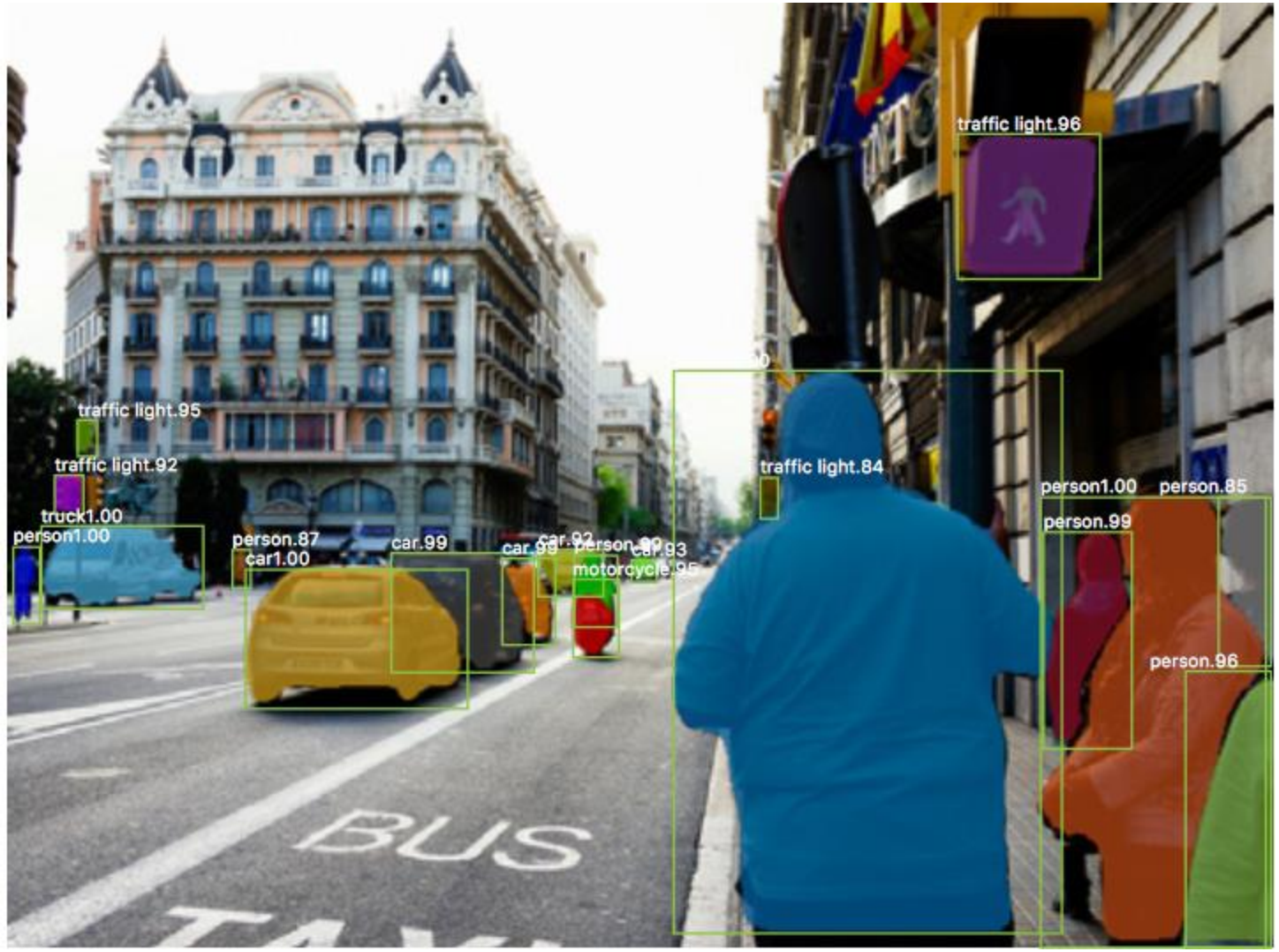
Similar to R-CNN but predicts mask *as well as* bounding box



Mask R-CNN

Similar to R-CNN but predicts mask as well as bounding box




traffic light.96
traffic light.95
traffic light.92
truck1.00
person1.00
person.87
car1.00
car.99
car.99
car.92
car.93
motorcycle.95
traffic light.84
person1.00
person.85
person.99
person.96
BUS

Mask R-CNN
Also does pose

He et al, "Mask R-CNN", arXiv 2017

Common Objects in COntext (COCO)

Dataset

80 objects
117,261 train/val images
902,435 object instances

New detection metric, mAP averaged over IOU [.5 - .95]

Segmentation masks for each instance

http://cocodataset.org/